

HackHustle



Submitted in partial fulfilment of the Degree of
Bachelor of Technology
in
Computer Science Engineering

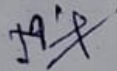
by
Bhoomika (2316372)
Charvi Khandelwal (2316376)
Divyanshi Chaudhary (2316387)
Fiza Qureshi (2316390)
Riza Qureshi (2316523)

Under the supervision of
Dr. Mainaz Faridi
Associate Professor
Department of Computer Science

DEPARTMENT OF COMPUTER SCIENCE
BANASTHALI VIDYAPITH, RAJASTHAN
2025-26

Certificate

This is to certify that the project work titled "**HACK HUSTLE**" submitted by **Bhoomika (2316372)**, **Charvi Khandelwal (2316376)**, **Divyanshi Chaudhary (2316387)**, **Fiza Qureshi (2316390)**, **Riza Qureshi (2316523)** in partial fulfillment for the award of degree of B. Tech in CSE of Banasthali Vidyapith has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.



Dr. Mainaz Faridi

Designation: Associate Professor

Place: Banasthali Vidyapith

Date: 8th April, 2026

Contents

1	Introduction	1
2	Software Requirement Specification (SRS)	2
2.1	Introduction	2
2.1.1	Purpose	2
2.1.2	Document Conventions	3
2.1.3	Intended Audience and Reading Suggestions	3
2.1.4	Scope	3
2.1.5	References	4
2.1.6	Overview	4
2.2	Overall Description	4
2.2.1	Product Perspective	4
2.2.2	Product Functions	4
2.2.3	User Classes and Characteristics	5
2.2.4	Operating Environment	5
2.2.5	Design and Implementation Constraints	6
2.2.6	User Documentation	6
2.2.7	Assumptions and Dependencies	7
2.3	External Interface Requirements	7
2.3.1	User Interfaces	7
2.3.2	Hardware Interfaces	29
2.3.3	Software Interfaces	30
2.4	System Features	30
2.4.1	User Registration and Authentication	30
2.4.2	Question Bank	30
2.4.3	Progress Tracking	30
2.4.4	Security and Access Control	31
2.5	Other Non-Functional Requirements	31
2.5.1	Performance Requirements	31
2.5.2	Safety Requirements	31
2.5.3	Security Requirements	31
2.5.4	Software Quality Attributes	31
2.5.5	Business Rules	31
3	Software Design Specification Document (SDS)	32
3.1	Introduction	32
3.1.1	Purpose	32
3.1.2	Scope	32

3.1.3	Overview	33
3.2	Structural Architecture Design	33
3.2.1	High-level Design Overview	33
3.2.2	Design Description of Components	34
3.3	Data Design	40
3.3.1	Database Description	40
3.3.2	ER Diagram	48
4	Coding	49
4.1	Practise Module:	49
4.2	Battle Module:	58
5	Testing	66
5.1	Types of Testing	66
6	User Interface	67
6.1	Description of Components	67
6.2	Screen Layouts	67
7	Appendix A: Glossary	71
7.1	Acronyms & Abbreviations	71
8	References	72

List of Figures

2.1	Student Use Case	8
2.2	Teacher Use Case	18
2.3	Guest User Use Case	23
2.4	Admin Use Case	25
3.1	System Architecture Diagram	33
3.2	Level-0 DFD	34
3.3	Level-1 DFD	35
3.4	Level-1 DFD	36
3.5	Level-1 DFD	37
3.6	Level-1 DFD	38
3.7	Activity Diagram	39
3.8	ER Diagram	48
6.1	The landing page of the platform	67
6.2	Login and Sign Up page of the platform	68
6.3	Practice page showing various subjects	68
6.4	Various topics being listed under a subject	68
6.5	The Interview page showing technical and behavioral interview options	69
6.6	Analysis after the interview showing the question asked, the answer provided and better answer	69
6.7	The Quiz Battle module showing modes to play a quiz, i.e. play with computer or a friend	69
6.8	Student's doubt to be displayed on the Academic Doubts page	70
6.9	The doubts asked are listed on the Faculty Dashboard	70

Chapter 1

Introduction

Hack Hustle is an integrated platform that bridges the gap between theoretical knowledge and practical skills by combining practice, assessment, interview preparation, and doubt resolution in one place. It helps students improve problem-solving, build conceptual clarity, and prepare for real-world technical challenges.

Unlike traditional methods, Hack Hustle offers interactive learning with continuous feedback and performance tracking. Students can apply knowledge, evaluate their progress, and improve over time, making it especially useful for placements, competitive exams, and skill development.

The platform consists of several key modules, each designed to target a specific aspect of learning. The practice module allows students to strengthen their understanding by solving problems across various subjects and topics. It provides categorised questions, enabling learners to focus on specific areas and gradually build expertise. The quiz module enables users to test their knowledge through timed assessments, which simulate real exam conditions.

Another crucial component is the interview module, which uses AI to simulate real interview scenarios, allowing students to participate in mock interviews where their responses are recorded. It helps improve confidence, communication, and approach to technical and behavioral questions. After each session, the platform provides AI-driven feedback, analyzing performance, highlighting strengths and weaknesses, and offering suggestions for improvement, enabling continuous growth and better preparation for real interviews.

Hack Hustle also includes a dedicated doubts module, which facilitates interaction between students and teachers, allowing them to ask questions and receive clear explanations from teachers. This ensures continuous learning without interruptions and enhances conceptual clarity.

Overall, Hack Hustle serves as a holistic platform that not only supports learning but also ensures continuous evaluation and improvement. By integrating practice, assessment, interview preparation, and doubt resolution with AI-driven insights, it empowers students to take control of their learning journey and prepares them effectively for academic and professional challenges.

Chapter 2

Software Requirement Specification (SRS)

2.1 Introduction

Software Requirement Specification (SRS) describes what our application will do. The following section provides an overview of the derived Software Requirements Specification (SRS) for the subject ‘**HackHustle**’.

HackHustle is an innovative education technology platform designed to enhance self-learning and assessment through MCQs, AI-powered interviews, multiplayer rapid-fire quizzes, faculty-assisted doubt clarification, and gamified learning. The platform scrapes knowledge from trusted sources, converts them into structured MCQs, and organises them subject-wise and module-wise. It provides students with real-time performance tracking, personalised feedback, and rewards to keep learning engaging.

2.1.1 Purpose

The primary purpose of HackHustle is to create an interactive ed-tech platform that enhances student learning through structured practice, collaboration, and personalised feedback. Traditional learning platforms provide theory and examples, but they lack assessments, real-time competition, gamified motivation, and doubt resolution.

The project aims to:

- **Simplify Self-Learning** – By topic-wise MCQs, students can practice systematically without searching for scattered resources.
- **Encourage Active Participation** – Through rapid-fire multiplayer quizzes, students engage in fast-paced, collaborative, and competitive learning.
- **Provide Personalised Feedback** – Using AI-powered mock interviews and performance tracking, students receive detailed insights into their strengths and weaknesses.
- **Resolve Doubts Efficiently** – The faculty support module ensures students get clarification on tough concepts, with a feedback-driven ranking system for quality assurance.

- **Motivate Through Gamification** – Virtual coins and rewards encourage consistency, achievement, and healthy competition among learners.

2.1.2 Document Conventions

This document follows a structured format based on the IEEE SRS standard.

The following conventions are applied:

- Headings and Subheadings are numbered (e.g., 1.1, 1.2, 2.1) for clarity and easy referencing.
- Bold text is used for section titles, key terms, and feature names.
- Diagrams follow standard UML notations (e.g., use case, ER diagrams) for consistency.
- All acronyms are summarised in the Acronyms and Abbreviations section.

2.1.3 Intended Audience and Reading Suggestions

The intended audience for this project primarily includes students and educators who seek to strengthen their knowledge and problem-solving abilities through structured practice and assessment. To gain the maximum benefit from this project, it is recommended that readers engage with the content progressively—beginning with foundational topics, advancing to subject-specific modules, and finally attempting assessments. Regular practice, consistent review of attempted questions, and the use of supplementary resources for clarification are suggested to ensure effective learning and long-term retention.

2.1.4 Scope

HackHustle is a comprehensive ed-tech platform with:

- **MCQ Solving**
 - Organising questions by subject and topics for targeted practice.
- **Progress Tracking**
 - Monitoring student’s topic completion percentage.
- **AI-Powered Mock Interviews**
 - Conducting simulated interviews with AI-driven questions.
 - Recording student responses and generating feedback reports.
- **Rapid Fire Quiz**
 - Allowing 2–5 students to compete in real-time quizzes.
 - Enforcing timers and instant question switching for a fast-paced experience.
 - Generating individual user reports at the end.

- **Doubt Resolution by Faculty**

- Enabling students to raise queries on specific questions.
- Routing doubts to faculty members for answers.
- Providing ranking to the faculty based on student feedback to maintain quality support.

- **Gamification Module**

- Rewarding students with virtual coins/points for achievements (e.g., winning quizzes, completing topics).

2.1.5 References

- IEEE Recommended Practice for Software Requirements Specification- IEEE Std. 830-1993.
- Pressman, Roger S., Software Engineering: A Practitioner's Approach, Fifth Edition, McGraw-Hill Publication, 2000.

2.1.6 Overview

The Overall Description section provides an overview of the product's purpose and functionality, outlining key requirements and how it will operate. The Specific Requirements section is for developers, detailing the product's functions. It specifies behaviours, inputs, outputs, and constraints, translating general goals into clear, actionable requirements for design, coding, and testing.

2.2 Overall Description

2.2.1 Product Perspective

'HackHustle' offers students a comprehensive platform for career preparation. The platform enables students to access practice mock interviews, take quizzes, and participate in interactive activities, such as rapid-fire rounds. Through this site, students can learn, practice, and monitor their progress in terms of accuracy and time management.

2.2.2 Product Functions

The major functions of the HackHustle platform are:

- **User Management** - Registration, login, and role-based access (Student, Teacher, Admin, Guest user).
- **Attempting Questions** - Students can attempt questions that are categorised by subjects and topics, making it easier for them to focus on specific areas.
- **Progress Tracking** - Students monitor their performance and current status.
- **Mock Interview** - AI-based interview simulation with feedback.

2.2.3 User Classes and Characteristics

Student

- Can register/log in to the platform.
- Access MCQs, quizzes, and assignments.
- Track progress, scores, and performance reports.
- Participate in mock interviews.
- Should have basic knowledge of English and computer usage.

Teachers

- Teacher Registration and authentication.
- Teachers get notified via email regarding doubts asked to them.
- Teachers are allowed to resolve doubts.
- Must be affiliated with a recognised institute.
- Should have subject expertise to clarify doubts.

Guest User

- Can browse the home page and user manual without registration.
- Can only access publicly available information on the platform.
- Encouraged to sign up for full access.

Admin

- Admin has full control over the platform.
- Must be an authorised person.
- Responsible for managing users, data, and overall system control.

2.2.4 Operating Environment

The system will operate in a standard client–server environment. On the client side, users will access the platform through modern web browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

On the server side, the application will run on a Windows or Linux-based operating system with a minimum hardware configuration of 8 GB RAM, and 100 GB storage to ensure smooth operation.

2.2.5 Design and Implementation Constraints

The development of the system is subject to several design and implementation constraints that must be carefully followed:

1. Technology Stack Constraints

- The backend is developed using Java as per the project requirement.
- The front-end is developed using React.js with Tailwind CSS to ensure a consistent and modern user interface.
- MySQL is used as the relational database.

2. Hardware and Performance Limitations

- The system must run efficiently on a minimum configuration of 8 GB RAM

3. Integration and Interface Constraints

- Compatibility with external APIs may impose restrictions.
- Browser compatibility is limited to Google Chrome, Mozilla Firefox, and Microsoft Edge (latest versions). Older browsers may not be supported.

4. Security and Privacy Considerations

- All user data (including quiz results and interview recordings) must comply with data privacy and security standards.
- Secure authentication protocols must be implemented to prevent unauthorised access.

5. Design Conventions and Standards

- The system initially supports two subjects only.
- Coding must follow Java coding conventions (Oracle guidelines) and React component best practices to ensure maintainability.
- Documentation standards (SDS, SRS) must be adhered to for project delivery and future maintenance.

6. Operational Constraints

- The system must support multiple students taking quizzes simultaneously.
- The system supports only the English language.

2.2.6 User Documentation

The system will be delivered with comprehensive user documentation to assist end users, administrators, and faculty in understanding and operating the platform effectively. A detailed guide describing system features, navigation, and step-by-step instructions for common tasks such as account registration, attempting questions, and participating in mock interviews.

2.2.7 Assumptions and Dependencies

The successful development and deployment of the system are based on the following assumptions and dependencies:

Assumptions

- Users should have access to a stable internet connection to use the platform effectively.
- Users should access the platform through modern web browsers (Chrome, Edge, Firefox).
- Students should use their own accounts and credentials without sharing login details.
- The number of concurrent users should remain within the expected system load limits.
- AI services or APIs used remain available and functional.
- The teacher must be affiliated with an authorized institution.

Dependencies

- Database availability: Continuous availability of the database for storing and retrieving data.
- Third-party APIs/Services: Availability of external APIs.
- Operating System/Browser Support: Dependence on users accessing the system through supported OS and browsers.

2.3 External Interface Requirements

2.3.1 User Interfaces

The user interface (UI) serves as the bridge between users and the system, enabling effective interaction based on their specific roles and needs. It is carefully designed to provide role-based access, ensuring that each type of user—such as students, teachers, and administrators—can easily access the features and functionalities relevant to them. For instance, students are provided with personalized dashboards where they can view assignments, track progress, attempt quizzes, and monitor their performance. On the other hand, teachers have access to dedicated interfaces that allow them to manage content, respond to student queries, and handle doubt clarification efficiently.

The UI focuses on delivering a seamless and intuitive experience by emphasizing simplicity, clarity, and consistency in design. Elements such as well-structured layouts, easy navigation menus, and clearly labeled options help users perform tasks without confusion. Additionally, the interface is designed to minimize complexity, allowing even users with basic technical knowledge to interact with the system comfortably.

2.3.1.1 Student

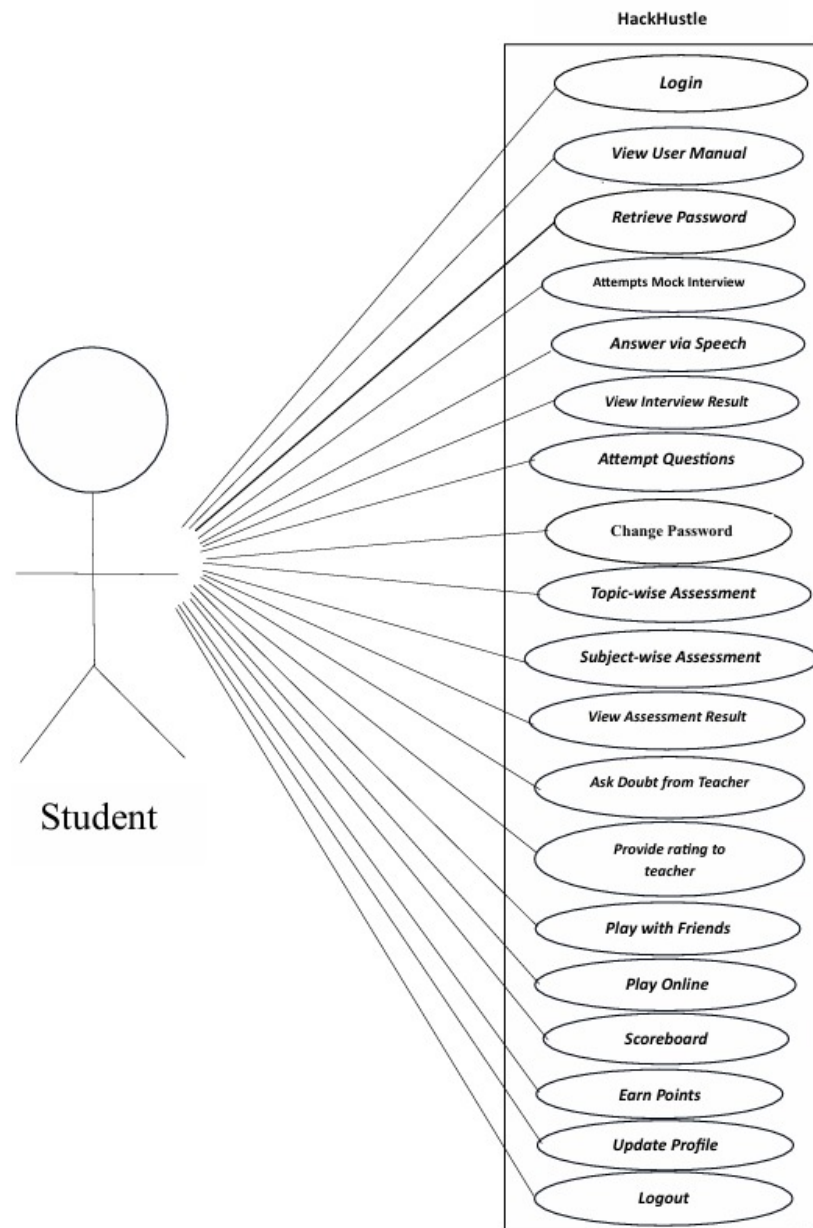


Figure 2.1: Student Use Case

2.3.1.1.1 Login

Introduction: Allows students to log in to the system.

Actors: Student.

Precondition: Student has valid credentials.

Postcondition: Student session established.

Flow of Events:

1. **Basic Flow:**

- Student enters username and password.

- System verifies credentials.
- Student is logged in and redirected to the dashboard.

2. **Alternative Flow:**

- AF1: Invalid credentials
 - (a) The system shows an error.
 - (b) Student retries login.

Use Case Relationships: All other modules.

2.3.1.1.2 Retrieve Password

Introduction: Allows students to reset their passwords.

Actors: Student.

Precondition: The Student account must exist in the system.

Postcondition: Password updated successfully.

Flow of Events:

1. **Basic Flow:**

- Student clicks “Forgot Password.”
- Enters registered email.
- System sends OTP/verification code.
- Student sets new password.
- System confirms update.

2. **Alternative Flow:**

- AF1: Invalid email
 - (a) Error shown.
 - (b) Student re-enters correct details.
- AF2: OTP expired- System prompts for resend.
- AF3: Weak password, System prompts for a stronger password.

2.3.1.1.3 Update Profile

Introduction: Enables students to modify profile details.

Actors: Student.

Precondition: Student logged in.

Postcondition: Profile updated.

Flow of Events:

1. **Basic Flow:**

- Student opens profile settings.
- Updates details (name, email, etc.).

- System validates and saves changes.

2. Alternative Flow:

- AF1: Invalid data
 - (a) System rejects.
 - (b) Student corrects and resubmits.
- AF2: Database error
 - (a) Failure shown.
 - (b) Student retries later.

Use Case Relationships:login.

2.3.1.1.4 Change Password

Introduction:Allows students to change their passwords.

Actors: Student.

Precondition: Student logged in with current password.

Postcondition: Password securely updated.

Flow of Events:

1. Basic Flow:

- Student navigates to change password.
- Enters current and new password.
- System validates and saves changes.
- Confirmation is displayed.

2. Alternative Flow:

- AF1: Incorrect current password
 - (a) System rejects update.
 - (b) Student retries with correct credentials.
- AF2: Weak password
 - (a) System prompts stronger password.
 - (b) Student resubmits.

Use Case Relationships:login.

2.3.1.1.5 View User Manual

Introduction: Enables students to review the user manual, which outlines the platform's functionalities and its operational process.

Actors: Student.

Precondition: The student must be on the home page.

Postcondition: User manual displayed successfully.

Flow of Events:

- Student navigates to the user manual.
- User manual is displayed.

Use Case Relationships: Login.

2.3.1.1.6 Attempts Mock Interview

Introduction: The Student begins a timed mock interview.

Actors: Student.

Precondition: The Student must be logged into the platform.

Postcondition: The question appears on the screen.

Flow of Events:

1. Basic Flow:

- Students access the mock interview.
- Student provides necessary details.
- Student confirms the selection.
- System displays the questions and starts the timer.

2. Alternative Flow:

- AF1: Invalid Session Start - If the system fails to initialise the session due to a technical error, the student is informed and asked to retry later.

Use Case Relationships: Answer via Speech

2.3.1.1.7 Answer via Speech

Introduction: Student answers verbally, system records and analyses.

Actors: Student.

Precondition: The student must have successfully started a timed interview session.

Postcondition: The student's audio response is recorded.

Flow of Events:

1. Basic Flow:

- A question is displayed on the student's screen.
- The system activates the microphone recording.
- The student verbally answers the question while being recorded.
- When the student finishes or the timer runs out, the response is saved.
- The system evaluates the response.

2. Alternative Flow:

- AF1: Student skips Question - If the student skips a question, the system loads the next question.

Special Requirements: Responses should be stored securely until analysis is complete.

Use Case Relationships: Result.

2.3.1.1.8 View Interview Result

Introduction: The system evaluates the correctness of answers.

Actors: Student.

Precondition:

1. The student must have completed answering questions.
2. The system must have access to recorded responses and transcripts.

Postcondition: Report is generated.

Flow of Events:

1. Basic Flow:

- The system retrieves the student's recorded responses.
- The responses are analysed.
- The system creates a report.
- The student views the evaluation report.

2. Alternative Flow:

- AF1: Incomplete Recording - the system evaluates only available data and marks missing sections.

Special Requirements: Data privacy must be ensured, no sharing of video/audio.

Use Case Relationships: N/A

2.3.1.1.9 Attempt Questions

Introduction: A student navigates through the MCQ practice module by selecting a subject, then a specific topic.

Actors: Student.

Precondition: The system must display a list of available subjects.

Postcondition: A curated set of MCQs appears.

Flow of Events:

1. Basic Flow:

- Student logs into the platform and navigates to the MCQ Practice.
- The system displays a list of subjects.
- Student selects a subject.
- The system displays available topics under that subject.
- Student selects a topic.
- The system retrieves and loads the MCQs based on the chosen subject and topic.
- The student proceeds to attempt the questions.

2. Alternative Flow:

- AF1: Internet/Server Issue - If questions fail to load, the system prompts the student to try again later.

Special Requirements: The system should load questions without delays.

Use Case Relationships: Topic-wise Assessment.

2.3.1.1.10 Topic-wise Assessment

Introduction: The student takes an assessment focused on a single topic from a subject.

Actors: Student.

Precondition: The student has already selected a subject and topic.

Postcondition: Student's answers are submitted and stored.

Flow of Events:

1. Basic Flow:

- The student chooses a topic from the chosen subject.
- System loads a set of MCQs.
- Student begins answering MCQs.
- System records each response.
- Results are processed.

2. Alternative Flow:

- AF1: Skipping Questions -
 - (a) The student may leave a question unanswered and move to the next one.
 - (b) The skipped question is marked "unattended."

Special Requirements: Results must be stored securely.

Use Case Relationships: Attempt Questions and Assessment Result.

2.3.1.1.11 Subject-wise Assessment

Introduction: The student takes a comprehensive test covering all topics within a selected subject.

Actors: Student.

Precondition: The student has already selected a subject.

Postcondition: Student's answers are recorded and stored.

Flow of Events:

1. Basic Flow:

- System displays a mixed-question set from all topics of the subject.
- Student begins answering MCQs sequentially.
- System records each response.
- Student receives the result after the assessment.

2. Alternative Flow:

- AF1: Skipping Questions
 - (a) The student may leave a question unanswered and move to the next one.
 - (b) The skipped question is marked “unattended.”

Use Case Relationships: Attempt Questions and Assessment Result.

2.3.1.1.12 View Assessment Result

Introduction: After completing an assessment, scores are visible to the student.

Actors: Student.

Precondition: Student must have completed an assessment.

Postcondition: Student receives a test report.

Flow of Events:

1. Basic Flow:

- Assessment is submitted.
- The system evaluates each question against correct answers.
- Student views the report.

2. Alternative Flow:

- AF1: Partial Submission (Incomplete Test) -If the student quits midway, a report is generated considering the left out question as unattempted.
- AF2: Network Failure on Submission - Answers are cached and synced once the connection is restored.

Use Case Relationships: Topic-wise Assessment and Subject-wise Assessment.

2.3.1.1.13 Ask Doubt from Teacher

Introduction: The student asks doubts to the teacher.

Actors: Student.

Precondition: There must be a teacher assigned for that subject.

Postcondition: Student receives a personalised response from the teacher.

Flow of Events:

1. Basic Flow:

- Student ask doubt from the teacher.
- Doubt is forwarded to the teacher.
- Teacher reviews and provides a detailed explanation.
- System delivers the teacher’s response to the student.

2. Alternative Flow:

- AF1: Teacher Offline - Student receives response once teacher is online.

Use Case Relationships: Provide Rating to Teacher.

2.3.1.1.14 Provide Rating to Teacher

Introduction: The students provide rating to the teachers.

Actors: Student.

Precondition:

1. The student must have previously asked a doubt from a teacher.
2. The teacher must have responded.

Postcondition: Rating is stored/updated in the system.

Flow of Events:

1. **Basic Flow:**

- Students view the teacher's response to their doubt.
- Student provides rating to the teacher.
- System records the rating and updates the teacher's profile.

2. **Alternative Flow:**

- AF1: Student Skips Rating - If the student chooses not to rate, the system allows them to continue without providing rating.

Use Case Relationships: Ask Doubt from Teacher

2.3.1.1.15 Play with Friends

Introduction: The student creates or join a quiz room using a unique code to compete with friends in real-time.

Precondition:

1. Student has access to the Rapid Quiz Battle Module.
2. At least one other friend joins the room to start the game.

Postcondition: Scores are generated.

Flow of Events:

1. **Basic Flow:**

- Student selects "Play with Friends" option.
- System offers two choices:
 - Create Room: Student generates a unique code and shares it with friends.
 - Join Room: Student enters a code to join an existing room.
- The first correct answer earns a point, and the question disappears for everyone.
- At the end, the system displays a summary screen showing individual scores and the winner.

2. Alternative Flow:

- AF1: Friend Fails to Join - If a student does not enter the room code in time, they cannot join after the quiz has started.
- AF2: Player Disconnects - If a participant disconnects, the game continues, and their progress is paused/saved.

Special Requirements: Real-time synchronisation of quiz questions and answers. **Use Case Relationships:** N/A.

2.3.1.1.16 Play Online

Introduction: The student plays real-time quiz battles against AI through the platform's play online feature.

Actors: Student.

Precondition: The student must be logged into the platform.

Postcondition: Scores are displayed.

Flow of Events:

- Student selects "Play Online" option.
- System starts the game session.
- both student and AI receive the same question simultaneously.
- First correct answer earns the point; the question disappears for all.
- At the end, a summary screen shows individual scores and the winner.

Use Case Relationships: N/A

2.3.1.1.17 Scoreboard

Introduction: After completing a quiz battle, the student can view a detailed summary of their performance.

Actor: Student.

Preconditions: Quiz battle must be completed.

Postconditions:

1. Students can view their score breakdown and accuracy.
2. The winner is highlighted.

Flow of Events:

1. Basic Flow:

- Quiz battle ends.
- System calculates scores based on:
 - Number of correct answers
 - Speed of answering

- First-to-answer bonus
- System generates the scoreboard.
- The summary screen displays:
 - Each participant's score
 - Winner of the match
 - Accuracy percentage
- Points are awarded.

2. Alternative Flow:

- AF1: Tie Situation - If two or more students score the same, the system declares a tie and highlights multiple winners.
- AF2: Incomplete Session - If a player disconnects, their partial performance is still recorded and displayed.

Use Case Relationships: Play with Friends, Play Online, Earn Points.

2.3.1.1.18 Earn Points

Introduction: This section defines how students collect points by engaging with various activities on the platform.

Actor: Student.

Preconditions: The student must perform a valid activity.

Postconditions: Points are credited to the student's profile.

Flow of Events:

1. Basic Flow:

- Student completes an activity
- System checks the point rules
- System calculates and allocates points.
- Updated points are reflected in the student's profile.

2. Alternative Flow:

- AF1: No Points Earned - If the activity is incomplete or abandoned, no points are awarded.

Use Case Relationships: All other modules.

2.3.1.1.19 Student Logout

Introduction: Allows students to securely log out of the platform.

Actors: Student.

Precondition: Student is logged in.

Postcondition: Student session ended.

Flow of Events:

1. Basic Flow:

- Student clicks "Logout."
- System terminates session.
- Home page is displayed.

2. Alternative Flow:

- AF1: Session timeout
 - (a) System auto-logs out the user.
 - (b) Student must re-login.

Use Case Relationships: All other modules.

2.3.1.2 Teacher

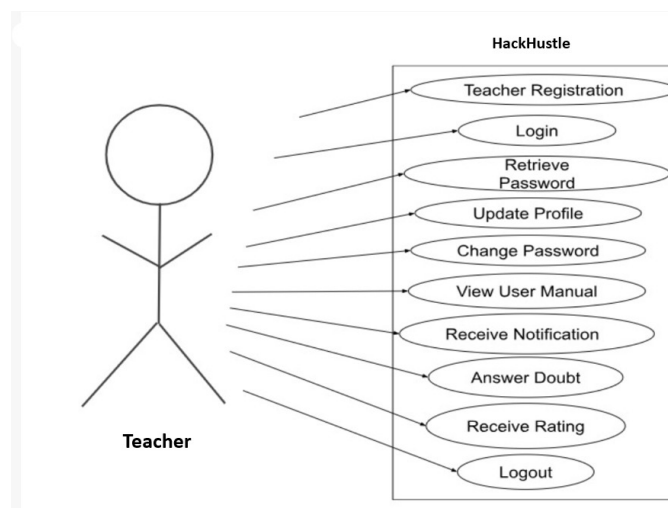


Figure 2.2: Teacher Use Case

2.3.1.2.1 Login

Introduction: Allows teachers to log in to the system.

Actors: Teacher.

Precondition: Teacher has valid credentials.

Postcondition: Teacher session established.

Flow of Events:

1. Basic Flow:

- Teacher enters username and password.
- System verifies credentials.

- Teacher is logged in and redirected to the dashboard.

2. Alternative Flow:

- AF1: Invalid credentials
 - (a) The system shows an error.
 - (b) Teacher retries login.

Use Case Relationships: All other modules.

2.3.1.2.2 Retrieve Password

Introduction: Allows the teacher to reset his/her passwords.

Actors: Teacher.

Precondition: The Teacher account must exist in the system.

Postcondition: Password updated successfully.

Flow of Events:

1. Basic Flow:

- Teacher clicks “Forgot Password.”
- Enters registered email.
- System sends OTP/verification code.
- Teacher sets new password.
- System confirms update.

2. Alternative Flow:

- AF1: Invalid email
 - (a) Error shown.
 - (b) Teacher re-enters correct details.
- AF2: OTP expired- System prompts for resend.
- AF3: Weak password, System prompts for a stronger password.

2.3.1.2.3 Update Profile

Introduction: Enables the teacher to modify profile details.

Actors: Teacher

Precondition: Teacher logged in.

Postcondition: Profile updated.

Flow of Events:

1. Basic Flow:

- Teacher opens profile settings.
- Updates details (name, email, etc.).
- System validates and saves changes.

2. Alternative Flow:

- AF1: Invalid data
 - (a) System rejects.
 - (b) Teacher corrects and resubmits.
- AF2: Database error
 - (a) Failure shown.
 - (b) Teacher retries later.

Use Case Relationships: login.

2.3.1.2.4 Change Password

Introduction: Allows teacher to change their passwords.

Actors: Teacher.

Precondition: Teacher logged in with current password.

Postcondition: Password securely updated.

Flow of Events:

1. Basic Flow:

- Teacher navigates to change password.
- Enters current and new password.
- System validates and saves changes.
- Confirmation is displayed.

2. Alternative Flow:

- AF1: Incorrect current password
 - (a) System rejects update.
 - (b) Teacher retries with correct credentials.
- AF2: Weak password
 - (a) System prompts stronger password.
 - (b) Teacher resubmits.

Use Case Relationships: login.

2.3.1.2.5 View User Manual

Introduction: Enables teachers to review the user manual, which outlines the platform's functionalities and its operational process.

Actors: Teacher.

Precondition: The teacher must be on the home page.

Postcondition: User manual displayed successfully.

Flow of Events:

- Teacher navigates to the user manual.
- User manual is displayed.

Use Case Relationships: Login

2.3.1.2.6 Teacher Registration

Introduction: Enables the teacher to register and obtain verification from the admin.

Actors: Teacher

Precondition: N/A

Postcondition: Teacher is verified and registered.

Flow of Events:

1. **Basic Flow:**

- Teacher navigates to the sign-up.
- Teacher enters details.
- Teacher registered successfully.

2. **Alternative Flow:**

- AF1: Invalid data
 - (a) System rejects.
 - (b) Teacher corrects and resubmits.

Use Case Relationships: Browse Home Page

2.3.1.2.7 Receive Notifications

Introduction: How a teacher receives notifications whenever a student asks a doubt.

Actors: Teacher.

Precondition: Teacher must be logged in.

Postcondition: Notification appears on the Teacher's Dashboard.

Flow of Events:

1. **Basic Flow:**

- A student asks a doubt.
- System generates a notification.
- Notification is displayed on the Teacher's dashboard.

2. **Alternative Flow:**

- AF1: Teacher is logged out, then the notification will be delivered after the next login.
- AF2: Network/server error, then notification delivery is delayed.

2.3.1.2.8 Answer Doubts

Introduction: How a teacher responds to a student's doubt from the notification panel.

Actors: Teacher.

Precondition: Teacher must have received a doubt notification.

Postcondition: Student receives Teacher's answer.

Flow of Events:

1. Basic Flow:

- Teacher clicks on the doubt notification.
- System displays the doubt details.
- Teacher enters the answer.
- System sends it to the student.

2. Alternative Flow:

AF1: Teacher provides an invalid/empty response, then System prompts correction.

- AF2: Teacher ignores notification, then Doubt remains pending.

Special Requirements: There must be a doubt for the teacher to answer. **Use Case Relationships:** Receive Notifications.

2.3.1.2.9 Receive Rating

Introduction: How a teacher receives ratings from students after answering doubts.

Actors: Teacher.

Precondition: Teacher must have submitted an answer.

Postcondition: Teacher's rating is stored in the system.

Flow of Events:

1. Basic Flow:

- Student reviews the answer.
- Student provides a rating.
- System updates the teacher's overall rating profile.

2. Alternative Flow:

- AF1: Student skips feedback, then no rating update occurs.
- AF2: Rating system error, then update is not stored.

Use Case Relationships: Answer Doubt.

2.3.1.2.10 Logout

Introduction: Allows teacher to securely log out of the platform.

Actors: Teacher.

Precondition: Teacher is logged in.

Postcondition: Teacher session ended.

Flow of Events:

1. Basic Flow:

- Teacher clicks "Logout."
- System terminates session.
- Home page is displayed.

2. Alternative Flow:

- AF1: Session timeout
 - (a) System auto-logs out the user.
 - (b) Teacher must re-login.

Use Case Relationships: All other modules.

2.3.1.3 Guest User

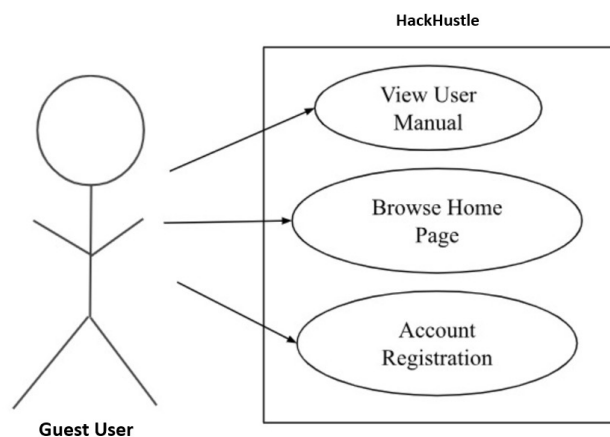


Figure 2.3: Guest User Use Case

2.3.1.3.1 View User Manual

Introduction: Enables the guest user to review the user manual, which outlines the platform's functionalities and its operational process.

Actors: Guest User.

Precondition: The guest user must be on the home page.

Postcondition: User manual displayed successfully.

Flow of Events:

- Guest user navigates to the user manual.
- User manual is displayed.

Use Case Relationships: Account Registration.

2.3.1.3.2 Browse Home Page

Introduction: Guest user can access home page, user manual without logging into the system.

Actors: Guest User

Precondition: User must not be logged in.

Postcondition: Home page content is displayed successfully.

Flow of Events:

1. **Basic Flow:**

- Guest searches for the platform.
- Home page is displayed to the guest user.

2. **Alternative Flow:**

- AF1: If the page does not exist then System shows “404 Page Not Found.”

Use Case Relationships: View User Manual, Login..

2.3.1.3.3 Account Registration

Introduction: Enables the guest user to register and create an account.

Actors: Guest User.

Precondition: The guest user must register.

Postcondition: Guest user is verified and registered.

Flow of Events:

1. **Basic Flow:**

- Guest user navigates to the sign-up.
- User enters details.
- User registered successfully.

2. **Alternative Flow:**

- AF1: Invalid data
 - (a) System rejects.
 - (b) User corrects and resubmits.

Use Case Relationships: Browse Home Page.

2.3.1.4 Admin

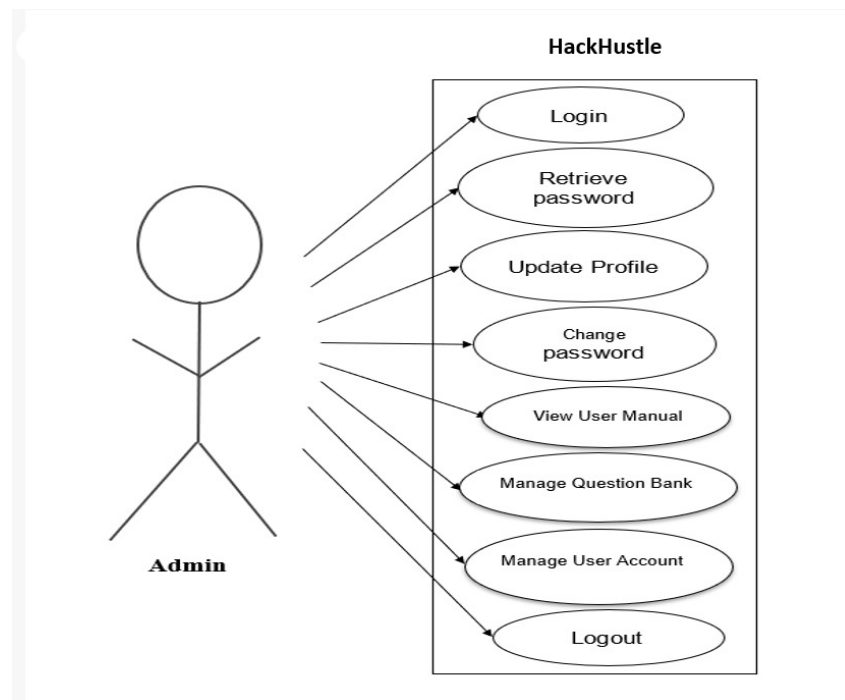


Figure 2.4: Admin Use Case

2.3.1.4.1 Login

Introduction: This feature allows the admin to log in to the system.

Actors: Admin.

Precondition: Admin has valid credentials.

Postcondition: Admin session established.

Flow of Events:

1. **Basic Flow:**

- Admin enters username and password.
- System verifies credentials.
- Admin is logged in and redirected to the dashboard.

2. **Alternative Flow:**

- AF1: Invalid credentials
 - (a) The system shows an error.
 - (b) Admin retries login.

Use Case Relationships: All other modules.

2.3.1.4.2 Retrieve Password

Introduction: Allows admin to reset their passwords.

Actors: Admin.

Precondition: The Admin account must exist in the system.

Postcondition: Password updated successfully.

Flow of Events:

1. **Basic Flow:**

- Admin clicks “Forgot Password.”
- Enters registered email.
- System sends OTP/verification code.
- Admin sets new password.
- System confirms update.

2. **Alternative Flow:**

- AF1: Invalid email
 - (a) Error shown.
 - (b) Admin re-enters correct details.
- AF2: OTP expired- System prompts for resend.
- AF3: Weak password, System prompts for a stronger password.

2.3.1.4.3 Update Profile

Introduction: Enables admin to modify profile details.

Actors: Admin.

Precondition: Admin must be logged in.

Postcondition: Profile updated.

Flow of Events:

1. **Basic Flow:**

- Admin opens profile settings.
- Updates details (name, email, etc.).
- System validates and saves changes.

2. **Alternative Flow:**

- AF1: Invalid data
 - (a) System rejects.
 - (b) Admin corrects and resubmits.
- AF2: Database error
 - (a) Failure shown.
 - (b) Admin retries later.

Use Case Relationships: login.

2.3.1.4.4 Change Password

Introduction: Allows admin to change their passwords.

Actors: Admin.

Precondition: Admin logged in with current password.

Postcondition: Password securely updated.

Flow of Events:

1. **Basic Flow:**

- Student navigates to change password.
- Enters current and new password.
- System validates and saves changes.
- Confirmation is displayed.

2. **Alternative Flow:**

- AF1: Incorrect current password
 - (a) System rejects update.
 - (b) Admin retries with correct credentials.
- AF2: Weak password
 - (a) System prompts for a stronger password.
 - (b) Admin resubmits.

Use Case Relationships: login.

2.3.1.4.5 View User Manual

Introduction: Enables the admin to review the user manual, which outlines the platform's functionalities and its operational process.

Actors: Admin.

Precondition: The admin must be on the home page.

Postcondition: User manual displayed successfully.

Flow of Events:

- Admin navigates to the user manual.
- User manual is displayed.

Use Case Relationships: Login.

2.3.1.4.6 Manage Question Bank

Introduction: Enables the admin to add, edit, or delete questions from the question bank.

Actors: Admin.

Precondition: The admin must be logged in with admin rights.

Postcondition: Question bank is modified.

Flow of Events:**1. Basic Flow:**

- Admin selects subject/topic.
- Adds, modifies, or deletes questions.
- System validates and updates the database.
- Confirmation is displayed.

2. Alternative Flow:

- AF1: Invalid input
 - (a) System rejects the entry.
 - (b) Admin corrects and resubmits.
- AF2: Database error
 - (a) Error message shown.
 - (b) Admin retries.

Use Case Relationships: MCQ modules.

2.3.1.4.7 Manage User Accounts

Introduction: Enables the Admin to manage user accounts, including validating and deactivating accounts.

Actors: Admin.

Precondition: User database active.

Postcondition: User account status is updated.

Flow of Events:**1. Basic Flow:**

- Admin opens the user panel.
- Validates or deactivates a user.
- Changes made are confirmed.

2. Alternative Flow:

- AF1: Duplicate detected
 - (a) System alerts the Admin.
 - (b) Admin reviews records.
 - (c) Cancels the request.
- AF2: Invalid details
 - (a) Entry is rejected.

Use Case Relationships: Student, Teacher, Guest User.

2.3.1.4.8 Logout

Introduction: Allows the admin to securely log out of the platform.

Actors: Admin.

Precondition: Admin is logged in.

Postcondition: Admin session ended.

Flow of Events:

1. Basic Flow:

- Admin clicks "Logout."
- System terminates session.
- Home page is displayed.

2. Alternative Flow:

- AF1: Session timeout
 - (a) System auto-logs out the user.
 - (b) The admin must re-login.

Use Case Relationships: All other modules.

2.3.2 Hardware Interfaces

1. Client Side

- Processor: Intel Core i3 or equivalent
- RAM: 2 GB minimum
- Storage (HDD/SSD): 200 MB free space
- Display: 1280x720 resolution or higher
- Network: Stable internet connection (1–2 Mbps)

2. Server Side

- Processor: Intel Core i3 or higher (dual-core)
- RAM: 4 GB
- Storage (HDD/SSD): 50–100 GB
- Network: Stable broadband connection (10 Mbps or more)

3. Developer Side

- Processor: Intel Core i3
- RAM: 4 GB
- Storage (HDD/SSD): 20–50 GB
- Operating System: Windows 10/11 or Linux

2.3.3 Software Interfaces

1. Client Side

- Operating System: Any modern operating system with network support (Windows 10/11, Linux, macOS).
- Browser: Any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).

2. Server Side

- Operating System: Windows Server 2019/2022 or Linux
- Web Server: Embedded Tomcat (provided by Spring Boot)
- Database Server: MySQL 8.0 (or any SQL-compliant server for production)

3. Developer Side

- Operating System: Windows 10/11 or Linux
- Browser: Any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari)
- Frontend: React JS (with VS Code or any code editor)
- Backend / IDE: Spring Boot (Java) using IntelliJ IDEA
- Database: MySQL 8.0

2.4 System Features

2.4.1 User Registration and Authentication

- The system shall allow new users (students/faculty) to create accounts with unique credentials.
- The system shall provide secure login using username/email and password.
- The system shall enforce password encryption and validation rules.
- The system shall allow password reset and profile update

2.4.2 Question Bank

- The system shall maintain a structured database of MCQs categorised by subject and sub-module.
- The system shall prevent duplication of questions in the database.

2.4.3 Progress Tracking

- The system shall track student progress for each subject and topic.
- The system shall display progress in with visual indicators.
- The system shall update progress after each activity.

2.4.4 Security and Access Control

- The system shall authenticate users before granting access to restricted features.
- The system shall ensure secure storage of user data and quiz results.

2.5 Other Non-Functional Requirements

2.5.1 Performance Requirements

The system shall deliver quiz generation, evaluation, and result display within 2–3 seconds under normal load. It must support concurrent access by multiple users without significant performance degradation. The platform should handle large question banks efficiently ensuring smooth operation.

2.5.2 Safety Requirements

The system shall ensure secure storage of user data to prevent loss or unauthorised access.

2.5.3 Security Requirements

- All communications between the client and server must be encrypted using HTTPS.
- Authentication tokens must be passed in headers to ensure secure communication.

2.5.4 Software Quality Attributes

The system shall meet the following quality attributes:

- The platform shall provide an intuitive and responsive interface, ensuring ease of use for both students and faculty.
- The codebase shall follow standard conventions, allowing future developers to update or enhance features with minimal effort.
- User data shall be protected using secure authentication and encryption mechanisms.

2.5.5 Business Rules

The user must have a basic understanding of the English language to effectively interact with the system.

Chapter 3

Software Design Specification Document (SDS)

3.1 Introduction

A Software Design Specification (SDS) document defines in detail what a system must accomplish. It translates the requirements into a clear and structured description that guides developers, testers, and other stakeholders. The diagrams provide an overview of the entire system, identifying the main components required for product development and their interfaces. The SDS ensures a shared understanding of the system's expected behaviour, setting the foundation for design and implementation while avoiding assumptions and ambiguity.

3.1.1 Purpose

The purpose of this Software Design Specification (SDS) is to provide a comprehensive blueprint for the design, structure, and workflow of the HackHustle platform. The SDS ensures that all technical, functional, and non-functional aspects of the software are clearly documented to guide both the ongoing development process and future maintenance.

3.1.2 Scope

The software product to be developed is HackHustle, a web-based platform designed to help students practice and improve their knowledge through features such as subject-wise MCQs, quizzes, progress tracking, and AI-driven mock interviews. The system will allow users to attempt categorised questions and track their performance. The application of Hack Hustle lies primarily in an academic environment, where students can enhance their preparation for technical interviews and assessments. The key objectives are to provide an engaging, structured practice environment, support personalised learning, and make performance evaluation more transparent. The overall goal is to create a useful, interactive platform that bridges the gap between theoretical knowledge and practical preparation.

3.1.3 Overview

The rest of this SDS outlines the structural architecture of Hack Hustle, including data designs and structure diagrams, user interface layouts, and testing types to be used. The document is organized to progress from high-level architecture and data flow to detailed interface design and testing strategies, ensuring a clear understanding of the system from creation to validation.

3.2 Structural Architecture Design

The architectural design of Hack Hustle defines the overall structure of the system and the interaction among its major components.

3.2.1 High-level Design Overview

At a high level, the system consists of a user interface for students and faculty, a backend server to manage business logic, a database to store questions, results, and user details, and an AI module to support mock interviews and question generation. These components communicate through structured data flows to ensure seamless operation.

A pictorial representation of the system architecture is presented below:

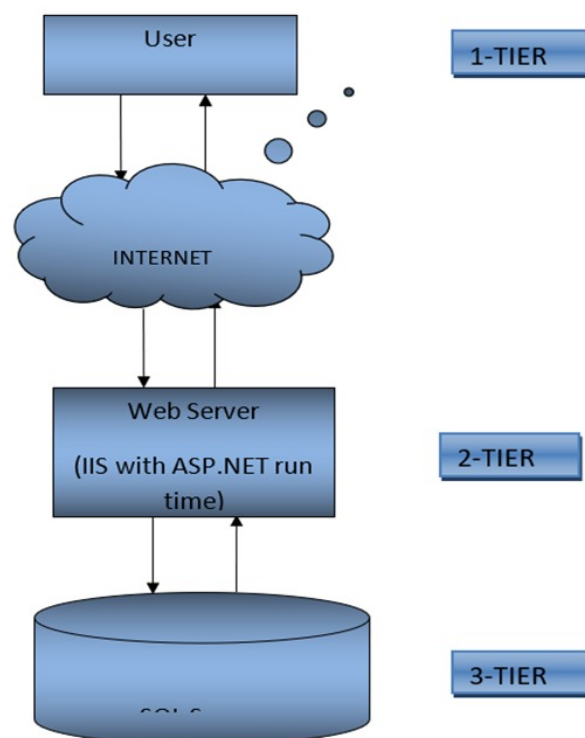


Figure 3.1: System Architecture Diagram

3.2.2 Design Description of Components

3.2.2.1 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of how data moves within a system. It shows the flow of information between external entities, processes, and data stores, highlighting how inputs are transformed into outputs through different system components.

3.2.2.1.1 Zero-Level Data Flow Diagram

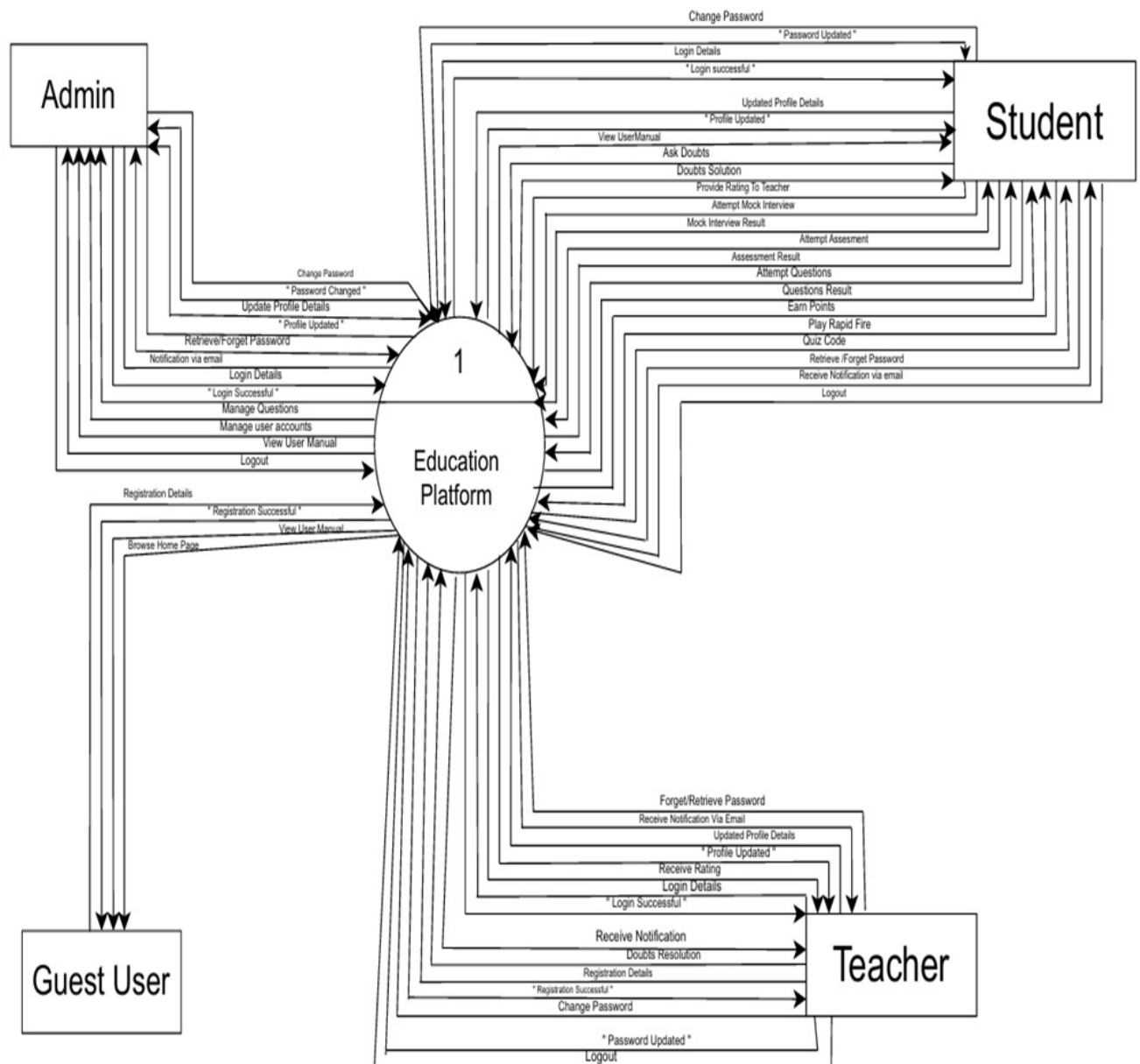


Figure 3.2: Level-0 DFD

3.2.2.1.2 First-Level Guest User Data Flow Diagram

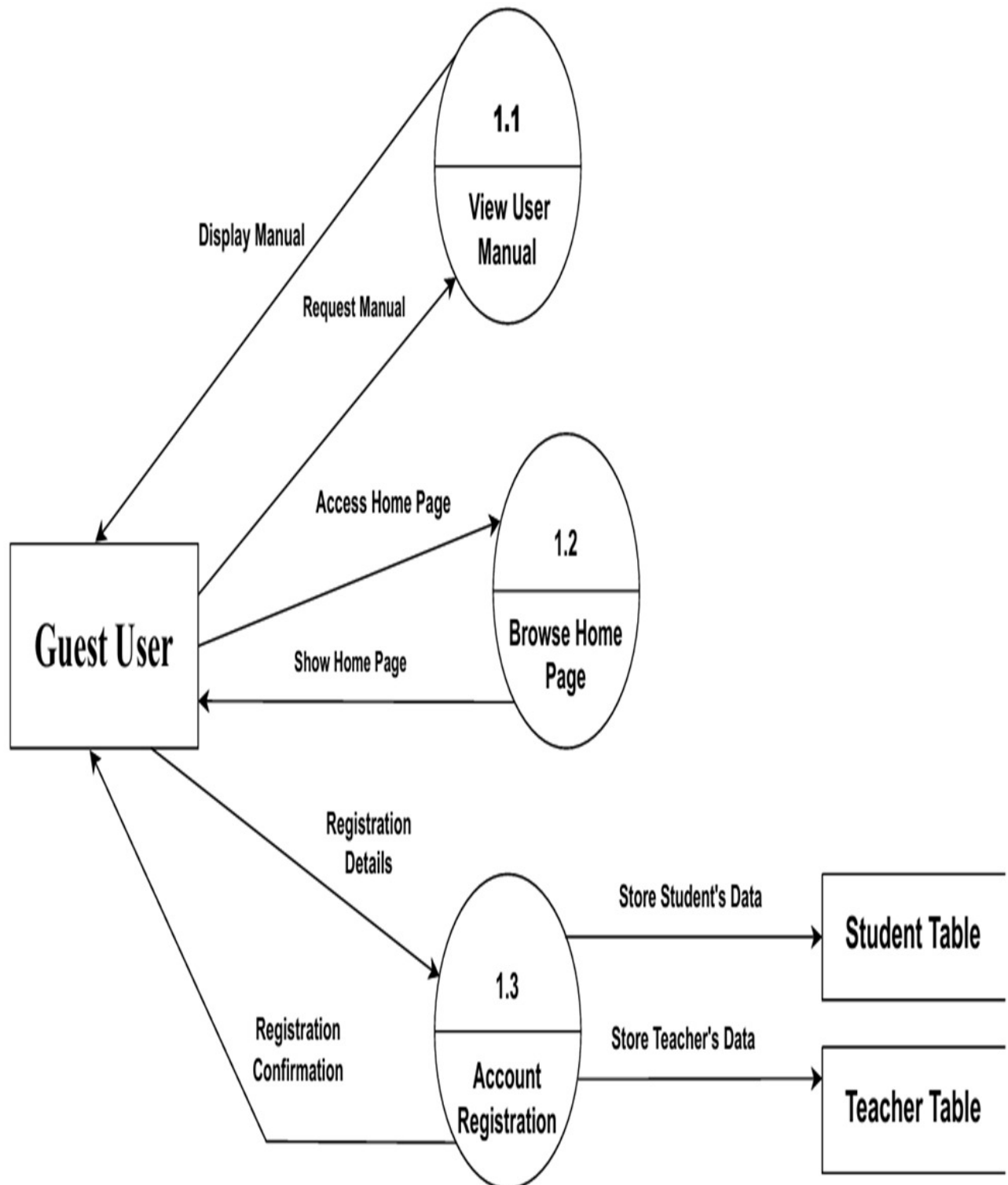


Figure 3.3: Level-1 DFD

3.2.2.1.3 First-Level Student Data Flow Diagram

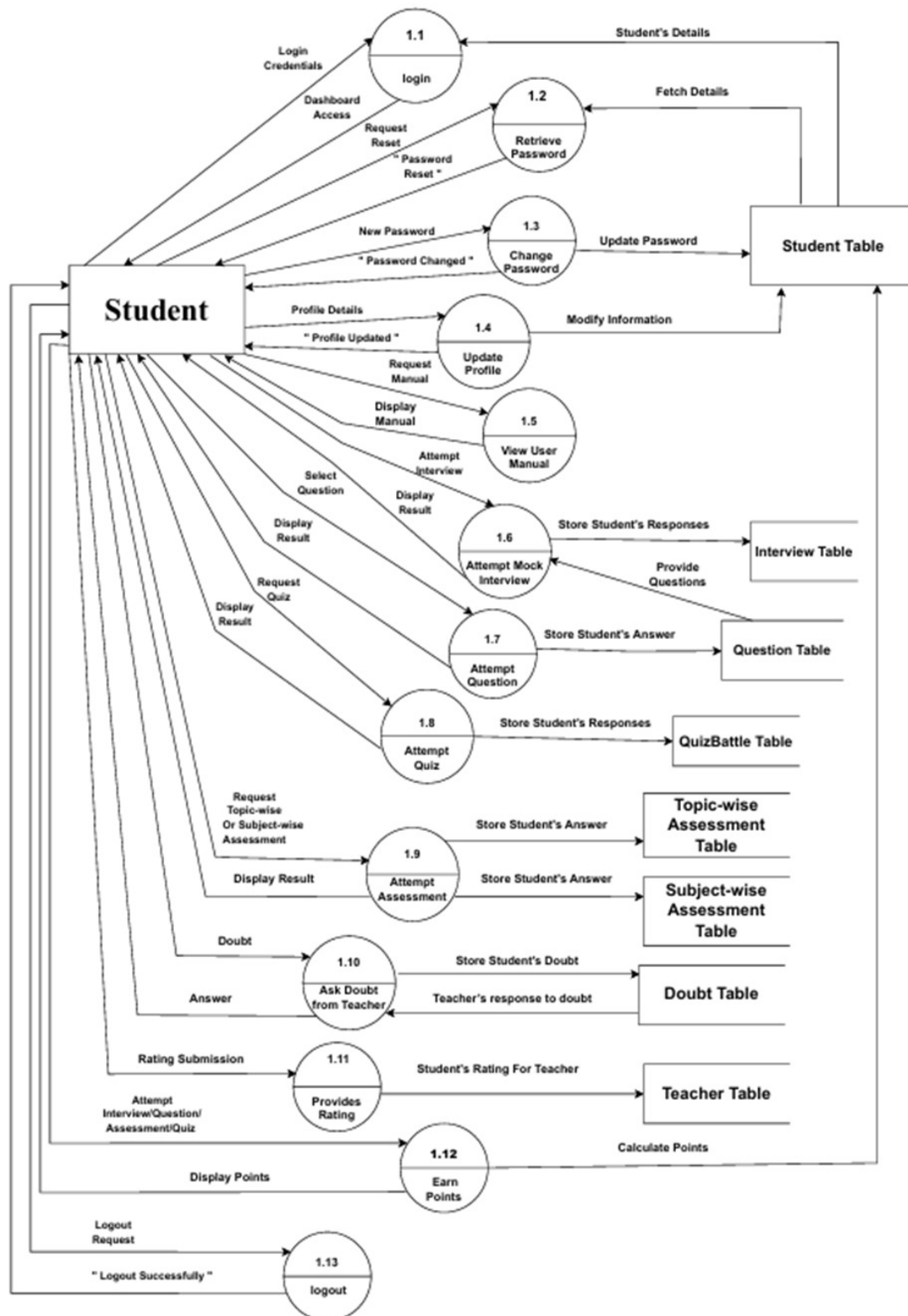


Figure 3.4: Level-1 DFD

3.2.2.1.4 First-Level Teacher Data Flow Diagram

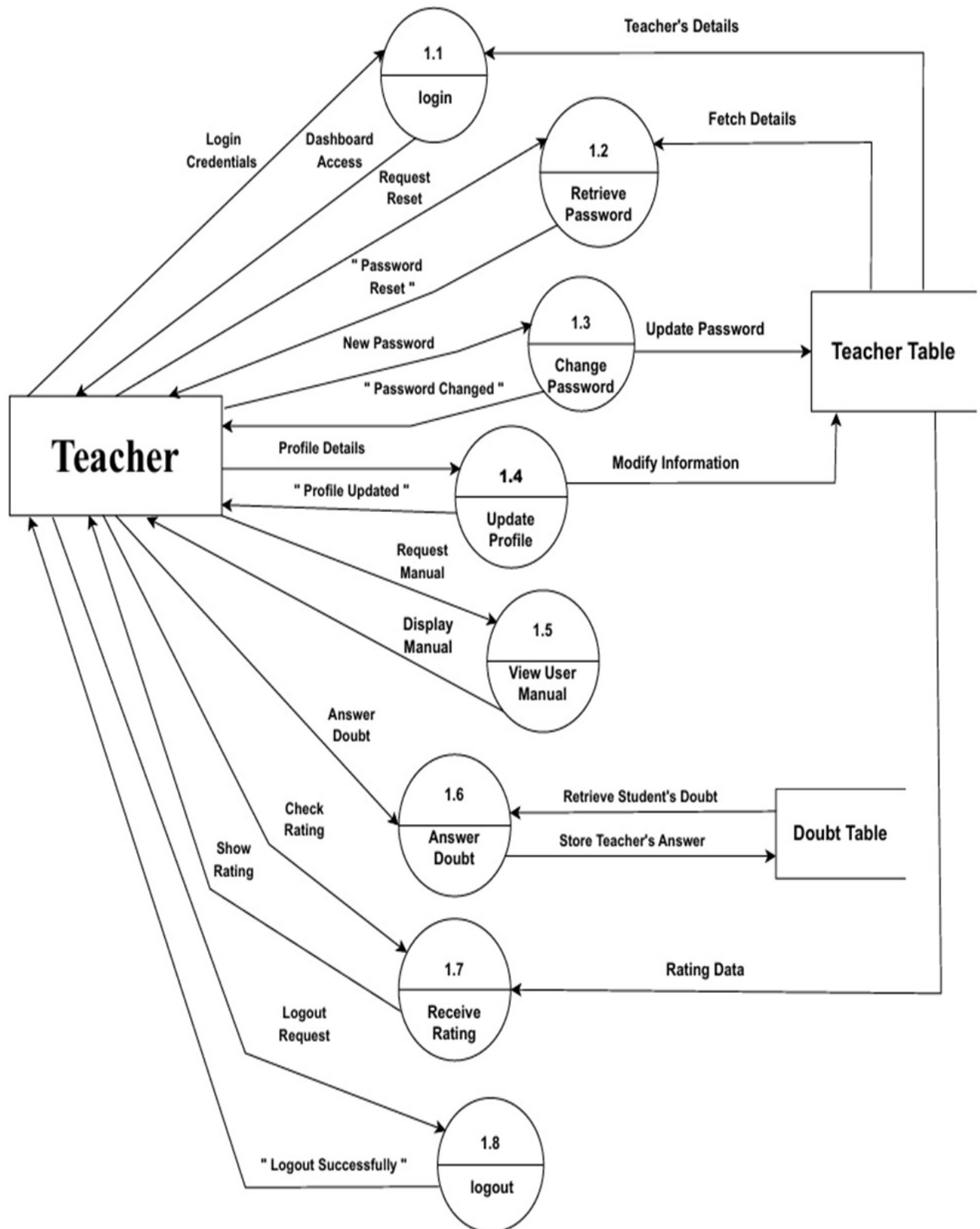


Figure 3.5: Level-1 DFD

3.2.2.1.5 First-Level Admin Data Flow Diagram

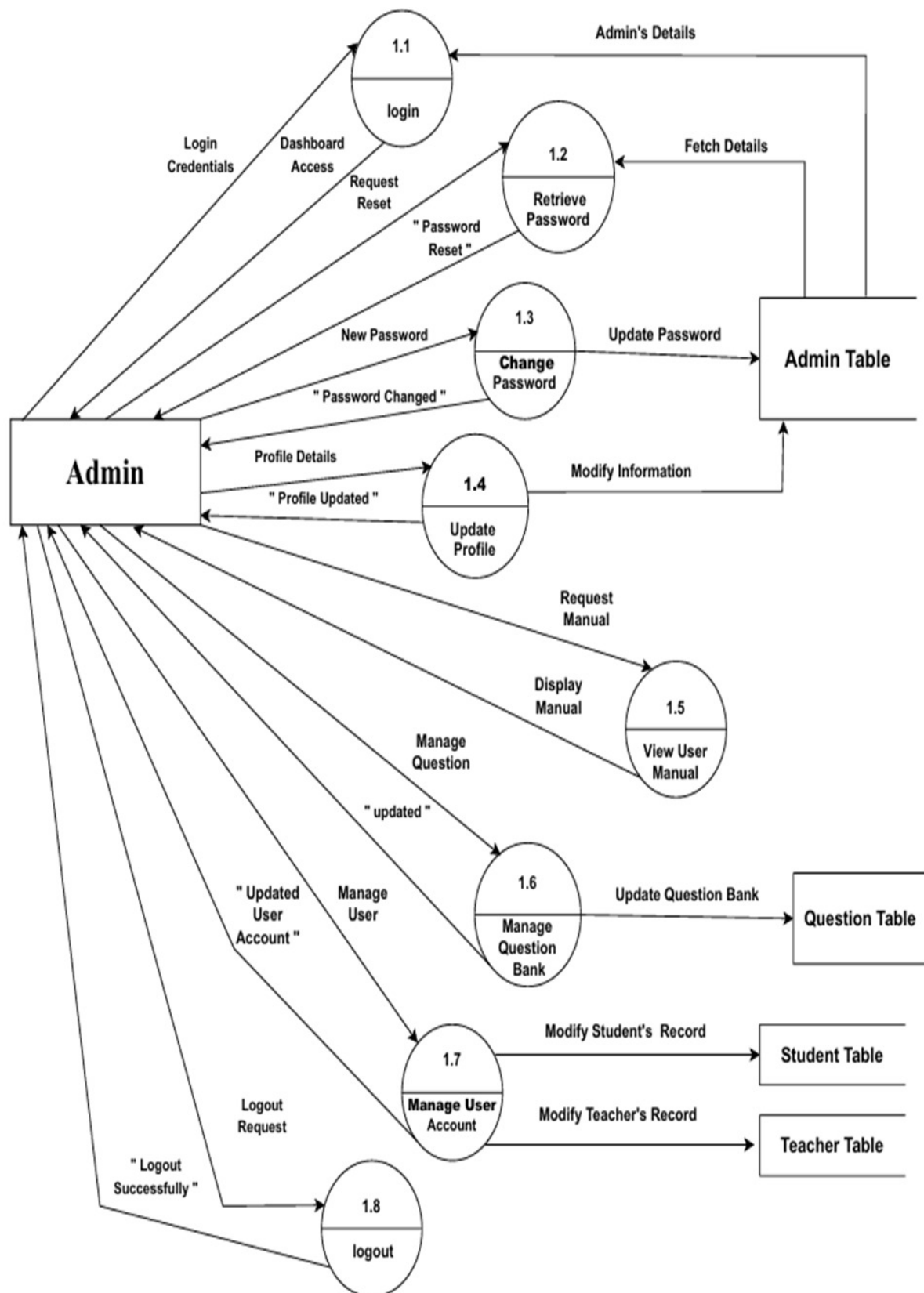


Figure 3.6: Level-1 DFD

3.2.2.2 Activity Diagram

An activity Diagram is a type of UML diagram that represents the workflow of a system or process. It shows the sequence of activities, decision points, and the flow of control from one activity to another, helping to visualise how a process starts, proceeds, and ends.

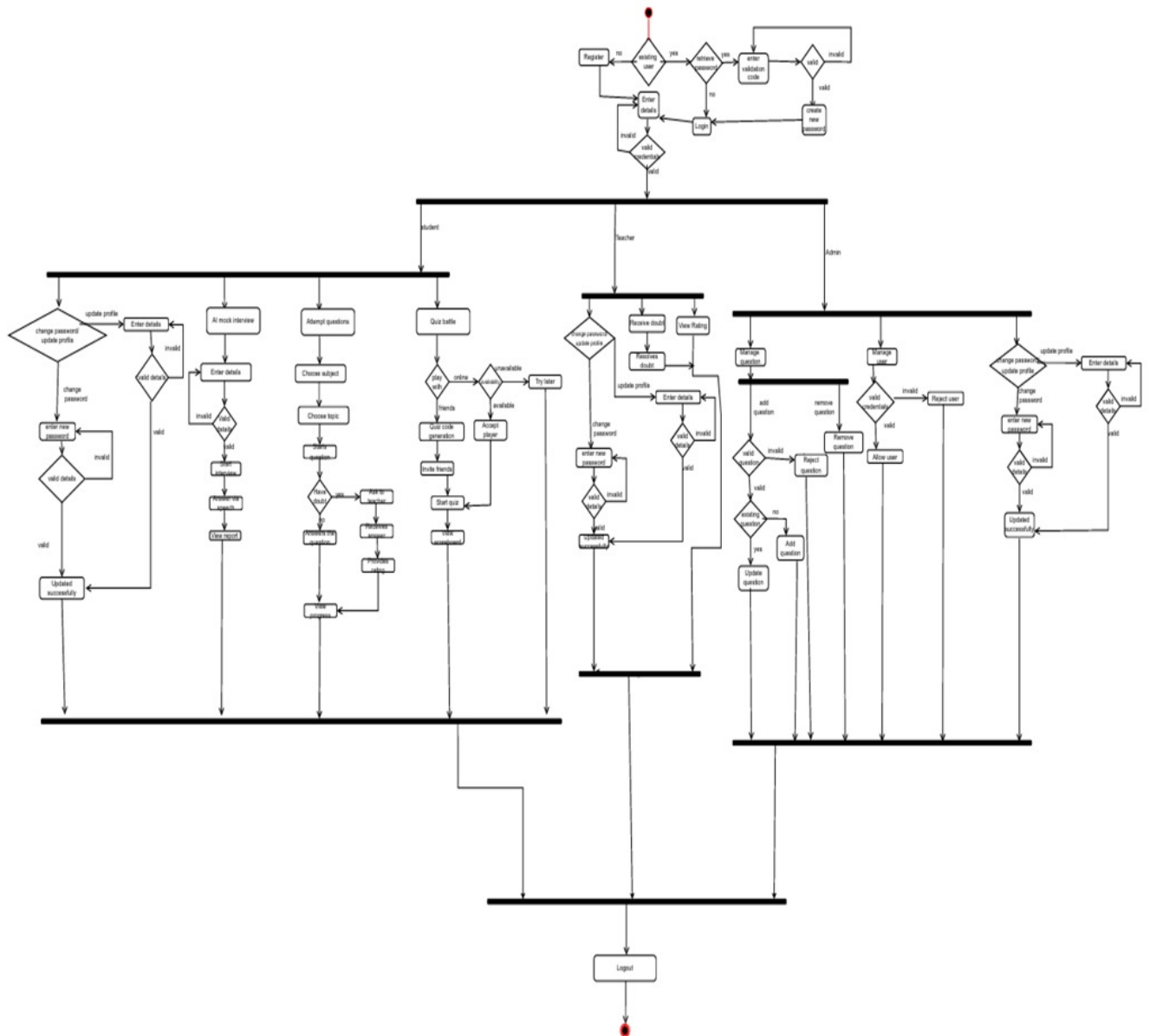


Figure 3.7: Activity Diagram

3.3 Data Design

3.3.1 Database Description

A huge amount of data has to be managed in the project, which is managed in the following way. All the data filled by the user is stored in various tables. Each table stores information related to a particular domain and only stores the information logically.

Student

Fields	Data Type	Description	Constraints
StudentID	INT	Unique identifier for each student (e.g., "1001").	Primary Key, NOT NULL
FirstName	VARCHAR(20)	Student's first name.	NOT NULL
LastName	VARCHAR(20)	Student's last name or surname.	NOT NULL
Email	VARCHAR(40)	Student's email address (must be unique, cannot be empty).	UNIQUE, NOT NULL
Password	VARCHAR(15)	Password for student login (should not be NULL).	NOT NULL
Points	INT	Points or credits assigned to a student (defaults to 0 initially).	DEFAULT 0
Quiz_attempted	int	Number of quiz attempted by the student.	DEFAULT 0

Doubt

Fields	Data Type	Description	Constraints
DoubtID	INT	Unique ID for each doubt or question (e.g., "1001").	Primary Key, NOT NULL
Status	CHAR(1)	Current status of the doubt. Example: 'U' = Unresolved, 'R' = Resolved.	DEFAULT 'U', NOT NULL
Date	DATE	Date when the doubt was raised.	NOT NULL
Query_asked	VARCHAR(200)	The question or doubt asked by the student.	NOT NULL
Answer_provided	VARCHAR(500)	The teacher's response or answer to the doubt.	NOT NULL
QuestionID	VARCHAR(5)	Primary key of the Question is associated with the student.	Foreign key, NOT NULL
StudentID	VARCHAR(5)	Primary key of the Student is associated with the student.	Foreign key, NOT NULL
Selected_subject	Varchar(255)	The subject related to the doubt.	
TeacherID	VARCHAR(5)	Primary key of the Teacher is associated with the student.	Foreign key, NOT NULL

Teacher

Fields	Data Type	Description	Constraints
TeacherID	INT	Unique identifier for each teacher (e.g., 1001).	Primary Key, NOT NULL
FirstName	VARCHAR(20)	Teacher's first name.	NOT NULL
LastName	VARCHAR(20)	Teacher's last name or surname.	NOT NULL
Email	VARCHAR(40)	The teacher's email address (it must be unique and cannot be empty).	UNIQUE, NOT NULL
Password	VARCHAR(15)	Password for teacher login (should not be NULL).	NOT NULL
Department	VARCHAR(40)	Department of the teacher	OPTIONAL
Designation	VARCHAR(40)	Designation of the teacher.	OPTIONAL
Experience	INT	Experience of the teacher in years.	OPTIONAL
Institute	VARCHAR(50)	Institute of the teacher	OPTIONAL
Subject	VARCHAR(20)	Subject taught by the teacher.	NOT NULL
Rating	TINYINT	Teacher's rating (from 0–5; defaults to 0).	DEFAULT 0, NOT NULL

Question

Fields	DataType	Description	Constraints
QuestionID	INT	Unique identifier for each question (e.g., 1001).	Primary Key, NOT NULL
QuestionText	VARCHAR(200)	The text of the question being asked.	NOT NULL
CorrectAnswer	VARCHAR(50)	The correct answer option.	NOT NULL
Option1	VARCHAR(200)	Text for the first answer choice	NOT NULL
Option2	VARCHAR(200)	Text for the second answer choice	NOT NULL
Option3	VARCHAR(200)	Text for the third answer choice	NOT NULL
Option4	VARCHAR(200)	Text for the fourth answer choice	NOT NULL
TopicID	INT	Primary key of the Topic is associated with the student.	Foreign key, NOT NULL

Topic

Fields	DataType	Description	Constraints
TopicID	INT	Unique identifier for each topic (e.g., 1001).	Primary Key, NOT NULL
TopicName	VARCHAR(20)	Name of the topic (e.g., "Stack").	NOT NULL
SubjectID	INT	Primary key of the Subject .	Foreign key, NOT NULL

SUBJECT

Fields	DataType	Description	Constraints
SubjectID	INT	Unique identifier for each subject (e.g., 1001).	Primary Key, NOT NULL
SubjectName	VARCHAR(20)	Name of the subject (e.g., "OOPS").	NOT NULL
Total_questions	INT	Total number of questions of that subject.	DEFAULT 0, NOT NULL

User Assessment

Fields	DataType	Description	Constraints
AssessmentID	INT	Unique ID for each subject-wise assessment (e.g., 1001).	Primary Key, NOT NULL
Date	DATE	Date when the assessment is taken by student.	NOT NULL
SubjectID	INT	Primary key of the Subject is associated with the student.	Foreign key, NOT NULL

StudentID	INT	Primary key of the Student is associated with the student.	Foreign key, NOT NULL
TopicID	INT	Primary key of the Question table is associated with the student.	Foreign key, NOT NULL
Assessment_score	INT	Store the result of user assessment of student.	Not Null

Interview

Fields	DataType	Description	Constraints
InterviewID	INT	Unique identifier for each interview session (e.g., 1001).	Primary Key, NOT NULL
Date	DATE	The date on which student takes mock interview.	NOT NULL
StudentID	INT	Primary key of the Student is associated with the student.	Foreign key, NOT NULL

InterviewResult

Fields	Data Type	Description	Constraints
InterviewResultID	INT	Unique identifier for each interview result record (e.g., 1001).	Primary Key, NOT NULL
ProvidedAnswer1	VARCHAR(200)	Answer given by user for question 1.	NOT NULL
ProvidedAnswer2	VARCHAR(200)	Answer given by user for question 2.	NOT NULL
ProvidedAnswer3	VARCHAR(200)	Answer given by user for question 3.	NOT NULL
BetterAnswer1	VARCHAR(200)	Suggested Answer for question 1.	NOT NULL
BetterAnswer2	VARCHAR(200)	Suggested Answer for question 2.	NOT NULL
BetterAnswer3	VARCHAR(200)	Suggested Answer for question 3.	NOT NULL
Question1	VARCHAR(200)	First question asked in interview	NOT NULL
Question2	VARCHAR(200)	Second question asked in the interview	NOT NULL
Question3	VARCHAR(200)	Third question asked in the interview	NOT NULL

InterviewID	INT	Primary key of the Interview table is associated with the student.	Foreign key, NOT NULL
InterviewScore	INT	Score obtained by the student	DEFAULT 0, NOT NULL

QuizBattle

Fields	DataType	Description	Constraints
QuizID	INT	Unique identifier for each quiz battle (e.g., 1001).	Primary Key, NOT NULL
PlayerNumber	INT	Number assigned to the player in the battle .	NOT NULL
QuizNumber	INT	Count of quiz attempted by student .	NOT NULL
QuizScore	INT	Points obtained by the student.	DEFAULT 0, NOT NULL
QuizStatus	VARCHAR(20)	Indicates whether the quiz is currently waiting or in the playing phase.	NOT NULL
StudentID	INT	Primary key of the Student is associated with the student.	Foreign key, NOT NULL

3.3.2 ER Diagram

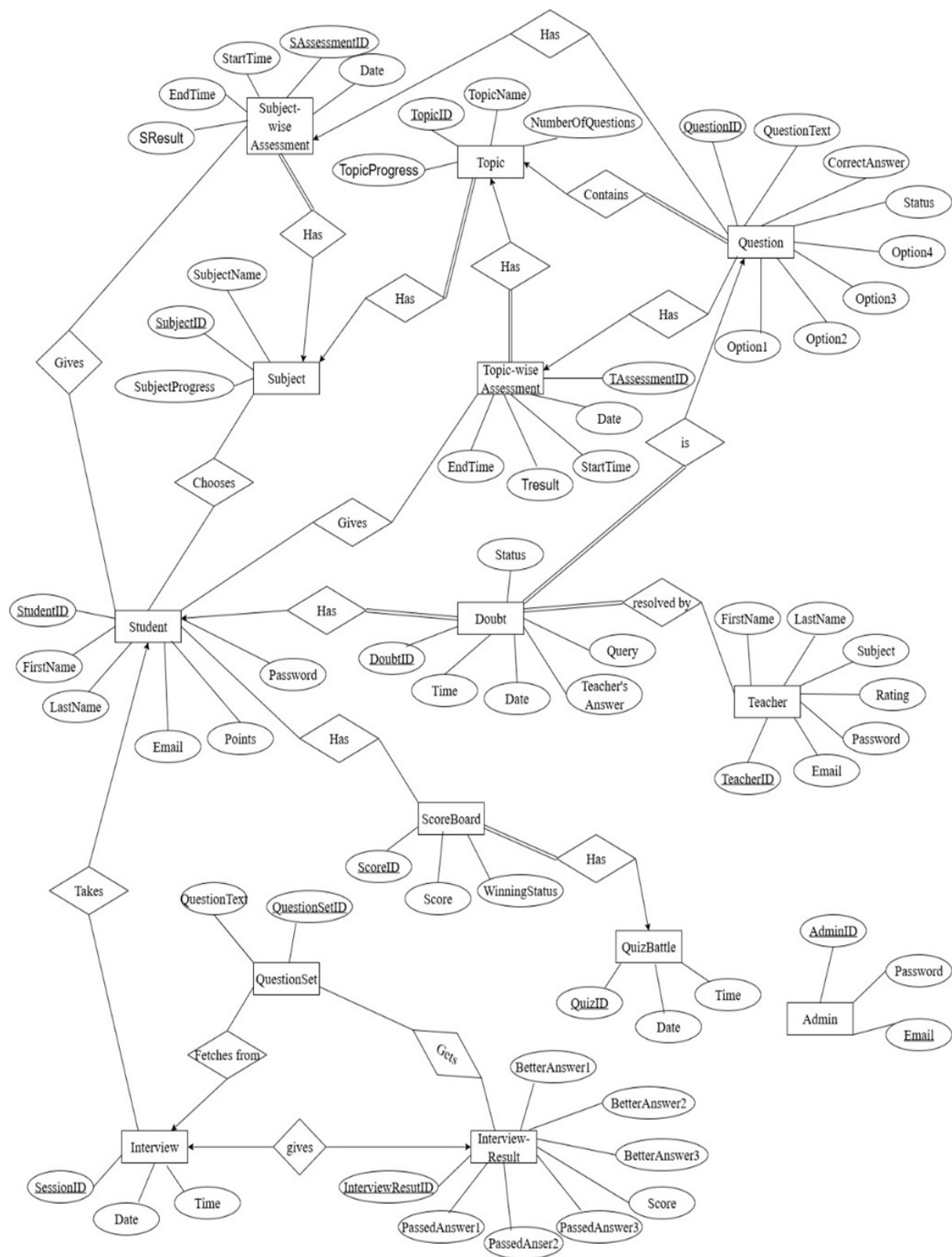


Figure 3.8: ER Diagram

Chapter 4

Coding

4.1 Practise Module:

```
1
2 import { useState, useEffect } from 'react';
3 import { Card, CardContent } from '@components/ui/card';
4 import { Button } from '@components/ui/button';
5 import { Badge } from '@components/ui/badge';
6 import { Dialog, DialogContent } from '@components/ui/dialog';
7 import {
8   BookOpen,
9   Code,
10  Database,
11  Play,
12  ChevronRight,
13  ChevronLeft,
14  Trophy
15 } from 'lucide-react';
16 import { useLocation } from 'react-router-dom';
17 import { Progress } from '@components/ui/progress';
18 import { CheckCircle2 } from 'lucide-react';
19 import PracticeQuiz from '../components/practice/PracticeQuiz';
20 import { questionService, QuestionDto } from '@services/questionsServices';
21
22 const user = JSON.parse(localStorage.getItem('user') || '{}');
23 const emailId: string = user?.emailId || '';
24
25
26 interface TopicConfig {
27   id: number;
28   name: string;
29 }
30 interface SubjectConfig {
31   id: number;
32   name: string;
```

```

33 topics: TopicConfig[];
34 }
35 const subjects: SubjectConfig[] = [
36 {
37   id: 1001,
38   name: 'DBMS',
39   topics: [
40     { id: 1001, name: 'Normalization' },
41     { id: 1002, name: 'Concurrency Control' },
42     { id: 1003, name: 'Deadlock' },
43     { id: 1004, name: 'Keys in DBMS' }
44   ]
45 },
46 {
47   id: 1002,
48   name: 'OOPS',
49   topics: [
50     { id: 1005, name: 'Concepts of OOPS' },
51     { id: 1006, name: 'Inheritance' },
52     { id: 1007, name: 'Abstraction' },
53     { id: 1008, name: 'Polymorphism' }
54   ]
55 },
56 {
57   id: 1003,
58   name: 'Data Structures',
59   topics: [
60     { id: 1009, name: 'Stack' },
61     { id: 1010, name: 'Linked List' },
62     { id: 1011, name: 'Queue' },
63     { id: 1012, name: 'Tree' },
64     { id: 1013, name: 'Heap' },
65     { id: 1014, name: 'Graphs' }
66   ]
67 }
68 ];
69 const splitIntoSets = (questions: QuestionDto[], size = 10) => {
70   const sets: QuestionDto[][] = [];
71   for (let i = 0; i < questions.length; i += size) {
72     sets.push(questions.slice(i, i + size));
73   }
74   return sets;
75 };
76 const Practice = () => {
77   const location = useLocation();
78   const [selectedSubject, setSelectedSubject] = useState<SubjectConfig | null>(null);
79   const [selectedTopic, setSelectedTopic] = useState<TopicConfig | null>(null);
80   const [questionSets, setQuestionSets] = useState<QuestionDto [][]>([]);

```

```

81  const [selectedSet, setSelectedSet] = useState(0);
82  const [startQuiz, setStartQuiz] = useState(false);
83  const [loading, setLoading] = useState(false);
84  const [assessmentScore, setAssessmentScore] = useState(0);
85  const handleSubmitAssessment = async () => {
86    setShowResult(true);
87    try {
88      await fetch('http://localhost:8080/api/assessments', {
89        method: 'POST',
90        headers: { 'Content-Type': 'application/json' },
91        body: JSON.stringify({
92          emailId: emailId,
93          topicID: selectedTopic?.id || 1015,
94          // subjectID: selectedSubject!.id,
95          subjectID: selectedSubject?.id,
96          assessmentScore
97        })
98      });
99      console.log("Data saved successfully");
100    } catch (err) {
101      console.error('Data save nahi hua (Backend Error):', err);
102    }
103  };
104  const [showResult, setShowResult] = useState(false);
105  const [topicProgress, setTopicProgress] = useState<Record<number, { solved: number,
↵ total: number }>>({});
106  const [isLoadingProgress, setIsLoadingProgress] = useState(false);
107  useEffect(() => {
108    if (selectedSubject && emailId) {
109      const fetchProgress = async () => {
110        setIsLoadingProgress(true);
111        try {
112          const response = await
↵ fetch(`http://localhost:8080/api/subjects/progress/${emailId}/${selectedSubject.id}`,
↵ {
113            cache: 'no-store'
114          });
115          if (response.ok) {
116            const data = await response.json();
117            const progressMap: Record<number, { solved: number, total: number }> = {};
118            data.forEach((item: any) => {
119              progressMap[item.topicId] = { solved: item.solvedQuestions, total:
↵ item.totalQuestions };
120            });
121            setTopicProgress(progressMap);
122          }
123        } catch (error) {
124

```



```

125     };
126     fetchProgress();
127   }
128   }, [selectedSubject, emailId, location.key]);
129   const [subjectProgress, setSubjectProgress] = useState<Record<number, { solved:
↪   number, total: number }>>({});
130   const [isLoadingSubjectProgress, setIsLoadingSubjectProgress] = useState(false);
131   useEffect(() => {
132     if (!selectedSubject && emailId) {
133       const fetchSubjectProgress = async () => {
134         setIsLoadingSubjectProgress(true);
135         try {
136           const response = await
↪   fetch(`http://localhost:8080/api/subjects/progress/subjects/${emailId}`, {
137             cache: 'no-store'
138           });
139           if (response.ok) {
140             const data = await response.json();
141             const progressMap: Record<number, { solved: number, total: number }> = {};
142             data.forEach((item: any) => {
143               progressMap[item.subjectId] = { solved: item.solvedQuestions, total:
↪   item.totalQuestions };
144             });
145             setSubjectProgress(progressMap);
146           }
147         } catch (error) {
148           console.error('Subject progress load nahi hui:', error);
149         }
150       };
151       fetchSubjectProgress();
152     }
153   }, [selectedSubject, emailId, location.key]);
154   const handleSelectSubject = (subject: SubjectConfig) => {
155     setSelectedSubject(subject);
156     setSelectedTopic(null);
157     setQuestionSets([]);
158     setStartQuiz(false);
159   };
160   const handleSelectTopic = async (topic: TopicConfig) => {
161     setSelectedTopic(topic);
162     setLoading(true);
163     try {
164       const data = await questionService.getQuestionsByTopic(topic.name);
165       setQuestionSets(splitIntoSets(data, 10));
166       setSelectedSet(0);
167     } catch (err) {
168       console.error(err);
169       setQuestionSets([]);

```

```

170     }
171   };
172   const handleTopicAssessment = async () => {
173     if (!selectedTopic) return;
174     setLoading(true);
175     try {
176       const data = await
↪ questionService.getTopicAssessmentQuestions(selectedTopic.name);
177       setQuestionSets([data]);
178       setSelectedSet(0);
179       setStartQuiz(true);
180     } catch (err) {
181       console.error(err);
182     }
183   };
184   const handleSubjectAssessment = async () => {
185     if (!selectedSubject) return;
186     setLoading(true);
187     try {
188       const data = await questionService.getQuestionsBySubject(selectedSubject.id);
189       setQuestionSets([data]);
190       setSelectedSet(0);
191       setStartQuiz(true);
192     } catch (err) {
193       console.error(err);
194     }
195   };
196   const handleBackToSubjects = () => {
197     setSelectedSubject(null);
198     setSelectedTopic(null);
199     setQuestionSets([]);
200     setStartQuiz(false);
201   };
202   const handleBackToTopics = () => {
203     setSelectedTopic(null);
204     setQuestionSets([]);
205     setStartQuiz(false);
206   };
207   if (startQuiz && selectedSubject) {
208     return (
209       <div className="min-h-screen bg-background pt-20 pb-24">
210         <PracticeQuiz
211           questions={questionSets[selectedSet] || []}
212           subjectName={selectedSubject.name}
213           topicName={selectedTopic?.name}
214           onBack={() => setStartQuiz(false)}
215           totalQuestionsInTopic={questionSets.flat().length}
216           onScoreCalculated={setAssessmentScore}

```

```

217         isAssessment={true}
218         subjectId={selectedSubject.id}
219         topicId={selectedTopic?.id || 1015}
220     />
221     <div className="p-4 flex justify-center">
222         <Button className="w-[240px]" onClick={handleSubmitAssessment}>
223             Submit Assessment
224         </Button>
225     </div>
226     <Dialog open={showResult} onOpenChange={setShowResult}>
227         <DialogContent className="bg-[#1a1a2e] border border-gray-800 text-white
↵ sm:max-w-md p-0 overflow-hidden">
228             <div className="flex flex-col items-center justify-center p-8 space-y-6">
229                 <div className="relative">
230                     <div className="absolute inset-0 bg-red-600 blur-2xl opacity-20
↵ rounded-full"></div>
231                     <div className="relative bg-[#2a2a3e] p-5 rounded-full border
↵ border-gray-700 shadow-xl">
232                         < Trophy className="h-10 w-10 text-red-500" />
233                     </div>
234                 </div>
235                 <div className="text-center">
236                     <h2 className="text-2xl font-bold">Assessment Submitted!</h2>
237                     <p className="text-gray-400 text-sm">
238                         Here is your final score
239                     </p>
240                 </div>
241                 <div className="bg-[#0f0f1a] w-full py-6 rounded-xl border
↵ border-gray-800 flex flex-col items-center">
242                     <span className="text-gray-500 text-xs uppercase font-bold mb-2">
243                         Total Score
244                     </span>
245                     <div className="flex items-baseline gap-2">
246                         <span className="text-5xl font-extrabold text-red-500">
247                             {assessmentScore}
248                         </span>
249                         <span className="text-xl text-gray-600">Points</span>
250                     </div>
251                 </div>
252                 <button
253                     onClick={() => setShowResult(false)}
254                     className="w-full py-3 rounded-lg bg-red-600 hover:bg-red-700
↵ text-white font-semibold"
255                     >
256                     Close Result
257                 </button>
258             </div>
259         </DialogContent>

```

```

260     </Dialog>
261   </div>
262   );
263 }
264   if (selectedSubject && selectedTopic) {
265   return (
266     <div className="min-h-screen bg-background pt-20 pb-16">
267       <div className="container mx-auto px-4 max-w-4xl">
268         <Button variant="outline" onClick={handleBackToTopics} className="mb-6">
269           <ChevronLeft className="w-4 h-4 mr-2" />
270           Back to Topics
271         </Button>
272         <Badge variant="secondary">{selectedSubject.name}</Badge>
273         <h1 className="text-3xl font-bold mt-2 mb-6">
274           {selectedTopic.name}
275         </h1>
276         {!loading && questionSets.length > 0 && (
277           <>
278             <div className="flex items-center justify-between flex-wrap gap-4 mb-6">
279               <div className="flex gap-3 flex-wrap">
280                 {questionSets.map((_, index) => (
281                   <Button
282                     key={index}
283                     variant={index === selectedSet ? 'default' : 'outline'}
284                     onClick={() => setSelectedSet(index)}
285                   >
286                     Set {index + 1}
287                   </Button>
288                 ))}
289               </div>
290               <Button variant="secondary" onClick={handleTopicAssessment}>
291                 Take Topic-wise Assessment
292               </Button>
293             </div>
294             <PracticeQuiz
295               questions={questionSets[selectedSet] || []}
296               topicName={selectedTopic.name}
297               subjectName={selectedSubject.name}
298               onBack={handleBackToTopics}
299               totalQuestionsInTopic={questionSets.flat().length}
300               isAssessment={false}
301               subjectId={selectedSubject.id}
302               topicId={selectedTopic.id}
303             />
304           </>
305         )}
306       </div>
307     </div>

```

```

308     );
309 }
310 if (selectedSubject && !selectedTopic && !startQuiz) {
311 return (
312   <div className="min-h-screen bg-background pt-20 pb-16">
313     <div className="container mx-auto px-4 max-w-4xl">
314       <Button variant="outline" onClick={handleBackToSubjects} className="mb-6">
315         <ChevronLeft className="w-4 h-4 mr-2" /> Back to Subjects
316       </Button>
317       <div className="mb-8">
318         <h1 className="text-4xl font-bold text-white mb-2">
319           {selectedSubject.name} Practice
320         </h1>
321         <p className="text-zinc-400">Track your progress and complete all
↪ topics.</p>
322       </div>
323       <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
324         {selectedSubject.topics.map((topic) => {
325           // API se data lo, warna default 0/10 dikhao
326           const prog = topicProgress[topic.id] || { solved: 0, total: 10 };
327           const progressPercentage = Math.round((prog.solved /
↪ Math.max(prog.total, 1)) * 100);
328           const isCompleted = prog.solved > 0 && prog.solved === prog.total;
329           return (
330             <Card
331               key={topic.id}
332               className="bg-[#09090b] border-zinc-800/80 hover:border-cyan-500/40
↪ hover:shadow-[0_0_25px_rgba(6,182,214,0.15)] transition-all duration-300
↪ cursor-pointer group rounded-xl relative overflow-hidden"
333               onClick={() => handleSelectTopic(topic)}
334             >
335               <div className="absolute inset-0 bg-gradient-to-br from-cyan-500/0 via-blue-500/0
↪ to-purple-500/0 group-hover:from-cyan-500/5 group-hover:to-blue-500/10
↪ transition-all duration-500 z-0" />
336               <CardContent className="p-5 flex flex-col justify-between h-full space-y-6 relative
↪ z-10">
337                 <div className="flex justify-between items-start">
338                   <div className="space-y-1.5">
339                     <h3 className="text-xl font-bold text-zinc-100 flex items-center gap-2
↪ group-hover:text-white transition-colors">
340                       {topic.name}
341                       {isCompleted && (
342                         <CheckCircle2 className="w-5 h-5 text-emerald-400
↪ drop-shadow-[0_0_8px_rgba(52,211,153,0.6)]" />
343                       )}
344                     </h3>
345                     <p className="text-sm text-zinc-500 font-medium group-hover:text-zinc-400
↪ transition-colors">

```

```

346         {isLoadingProgress ? (
347             <span className="animate-pulse text-cyan-500/70">Fetching data...</span>
348         ) : (
349             `${prog.solved} / ${prog.total} Problems`
350         )}
351     </p>
352 </div>
353     <div className="w-8 h-8 rounded-full bg-zinc-900 border border-zinc-800 flex
↪ items-center justify-center group-hover:bg-cyan-500/10
↪ group-hover:border-cyan-500/40 group-hover:shadow-[0_0_10px_rgba(6,182,214,0.2)]
↪ transition-all duration-300">
354         <ChevronRight className="w-4 h-4 text-zinc-600 group-hover:text-cyan-400
↪ transition-transform group-hover:translate-x-0.5" />
355     </div>
356 </div>
357 <div className="space-y-2">
358     <div className="flex justify-between text-xs font-bold tracking-wider
↪ uppercase">
359         <span className="text-zinc-600">Progress</span>
360         <span className={isCompleted ? "text-emerald-400
↪ drop-shadow-[0_0_5px_rgba(52,211,153,0.5)]" : "text-cyan-400
↪ drop-shadow-[0_0_5px_rgba(6,182,214,0.5)]"}>
361             {progressPercentage}%
362         </span>
363     </div>
364     <Progress
365         value={progressPercentage}
366         className="h-2 bg-zinc-900 border border-zinc-800/50
367         [>div]:bg-gradient-to-r [>div]:from-cyan-400 [>div]:to-blue-600
368         [>div]:shadow-[0_0_10px_rgba(6,182,214,0.6)]"
369     />
370 </div>
371 </CardContent>
372 </Card>
373     );
374   })}
375 </div>
376 <Button
377     className="w-full mt-10"
378     variant="secondary"
379     onClick={handleSubjectAssessment}
380 >
381     Start {selectedSubject.name}'s Final Assessment
382 </Button>
383 </div>
384 </div>
385 );
386 }

```

4.2 Battle Module:

```
1   import { useState, useEffect } from 'react';
2   import { Card } from '@components/ui/card';
3   import { Button } from '@components/ui/button';
4   import { Badge } from '@components/ui/badge';
5   import { Progress } from '@components/ui/progress';
6   import {
7     Trophy, Users, Zap, Clock, Target, Swords, Play,
8     CheckCircle2, XCircle, Medal
9   } from 'lucide-react';
10  import { battleQuestions, Question } from '@data/questions';
11  import { BattleReport } from '@components/reports/BattleReport';
12  import { BattleLobby, JoinBattleLobby } from './BattleLobby';
13  import { useRef } from 'react';
14  import {
15    connectBattleSocket,
16    sendJoinBattle,
17    sendAnswer,
18    disconnectBattleSocket
19  } from '@services/battleSocket';
20  import { useAuth } from '@contexts/AuthContext';
21  import { useNavigate } from 'react-router-dom';
22  import { ComputerBattle } from './ComputerBattle';
23  const getOptionsArray = (q: Question): string[] => {
24    return [q.option1, q.option2, q.option3, q.option4];
25  };
26  const battleModes = [
27    { id: 1, title: 'Quick Battle', description: 'Jump into a match with random
    ↪ players', icon: Zap, color: 'bg-yellow-500', maxPlayers: 5, duration: '5 minutes',
    ↪ questions: 20 },
28    { id: 2, title: 'Private Room', description: 'Create or join a room with friends',
    ↪ icon: Users, color: 'bg-blue-500', maxPlayers: 8, duration: '10 minutes',
    ↪ questions: 30 },
29  ];
30  const Battle = () => {
31    const [gameMode, setGameMode] = useState<'menu' | 'lobby' | 'join' | 'game' |
    ↪ 'report'>('menu');
32    const [currentQuestion, setCurrentQuestion] = useState(0);
33    const [selectedAnswers, setSelectedAnswers] = useState<{ [key: number]: number
    ↪ }>({});
34    const [showResults, setShowResults] = useState<{ [key: number]: boolean }>({});
35    const [timeLeft, setTimeLeft] = useState(600);
36    const [isGameActive, setIsGameActive] = useState(false);
37    const [startTime, setStartTime] = useState<number>(0);
38    const [roomCode, setRoomCode] = useState<string>('');
39    const [isHost, setIsHost] = useState(false);
```

```

40  const [selectedMode, setSelectedMode] = useState<number>(1);
41  const [players, setPlayers] = useState<number[]>([]);
42  const [dbQuestions, setDbQuestions] = useState<Question[]>([]);
43  const [liveScores, setLiveScores] = useState<{ [key: number]: number }>({});
44  const [masterAnswers, setMasterAnswers] = useState<{ [key: number]: number }>({});
45  const { user } = useAuth();
46  const navigate = useNavigate();
47  const studentEmail = user?.emailId;
48  const studentId = Number(user?.student_id);
49  console.log("User Info:", { studentEmail, studentId, user });
50  const [winnerId, setWinnerId] = useState<number | null>(null);
51  const currentQuestions = dbQuestions.length > 0 ? dbQuestions : battleQuestions;
52  const [isComputerMode, setIsComputerMode] = useState(false);
53  const scoresRef = useRef<{ [key: number]: number }>({});
54  const createBattleEntry = async (email: string, status: string = "COMPLETED", score:
↳  number = 0) => {
55    if (!roomCode) return;
56    try {
57      const numericQuizId = parseInt(roomCode.replace(/\D/g, "")) || 1;
58      const pNumber = players.indexOf(studentId) + 1 || 1;
59      const battleData = {
60        quizNumber: numericQuizId,
61        playerNumber: pNumber,
62        studentEmail: email,
63        quizScore: score,
64        status: status
65      };
66      const response = await fetch('http://localhost:8080/api/battle/create', {
67        method: 'POST',
68        headers: { 'Content-Type': 'application/json' },
69        body: JSON.stringify(battleData)
70      });
71      if (response.ok) console.log(` Entry saved successfully!`);
72    } catch (error) {
73      console.error("Final Submission Failed:", error);
74    }
75  };
76  const startBattle = (modeId: number) => {
77    setGameMode('game');
78    setIsGameActive(true);
79    setCurrentQuestion(0);
80    setSelectedAnswers({});
81    setShowResults({});
82    setTimeLeft(modeId === 2 ? 600 : 300);
83    setStartTime(Date.now());
84  };
85  useEffect(() => {
86    if (isGameActive && timeLeft > 0) {

```



```

87     const timer = setInterval(() => setTimeLeft((prev) => prev - 1), 1000);
88     return () => clearInterval(timer);
89   } else if (timeLeft === 0 && isGameActive) {
90     handleFinishGame();
91   }
92 }, [isGameActive, timeLeft]);
93 /* ----- SYNC LOGIC (FASTEST FINGER) ----- */
94 useEffect(() => {
95   if (gameMode === 'lobby' && roomCode) {
96     connectBattleSocket(roomCode, (data) => {
97       if (data === 'START' || data.status === 'START') {
98         startBattle(selectedMode);
99       } else if (data.battleCode) {
100         setPlayers(data.players || []);
101         if (data.questions) setDbQuestions(data.questions);
102         if (data.scores) {
103           console.log("Updated Scores from Server:", data.scores);
104           console.log("Full Scores Map:", JSON.stringify(data.scores));
105           setLiveScores(data.scores);
106           scoresRef.current = data.scores;
107         }
108         if (typeof data.currentQuestionIndex === 'number') {
109           if (data.currentQuestionIndex >= 10) {
110             handleFinishGame();
111           } else if (data.currentQuestionIndex !== currentQuestion){
112             setCurrentQuestion(data.currentQuestionIndex);
113             setShowResults({});
114             setSelectedAnswers({});
115           }
116         }
117       }
118     });
119     sendJoinBattle(roomCode, studentEmail!);
120   }
121   return () => {
122     if (gameMode !== 'lobby' && gameMode !== 'game') {
123       disconnectBattleSocket();
124     }
125   };
126 }, [gameMode, roomCode, currentQuestion]);
127 const handleInvite = async () => {
128   try {
129     const response = await
↵ fetch(`http://localhost:8080/api/battle/create/${studentEmail}`, { method: 'POST'
↵   });
130     const code = await response.text();
131     setRoomCode(code);
132     setIsHost(true);

```

```

133     setGameMode('lobby');
134   } catch (error) { console.error("Invite Error:", error); }
135 };
136 const handleJoinWithCode = async (code: string) => {
137   try {
138     await fetch(`http://localhost:8080/api/battle/join/${code}/${studentEmail}`, {
↪    method: 'POST' });
139     setRoomCode(code);
140     setIsHost(false);
141     setGameMode('lobby');
142   } catch (error) { console.error("Join Error:", error); }
143 };
144 const handleAnswer = (answerIndex: number) => {
145   const question = currentQuestions[currentQuestion];
146   const questionId = question.questionID;
147   const options = getOptionsArray(question);
148   const selectedOptionText = options[answerIndex];
149   setSelectedAnswers(prev => ({ ...prev, [questionId]: answerIndex }));
150   setShowResults(prev => ({ ...prev, [questionId]: true }));
151   setMasterAnswers(prev => ({ ...prev, [questionId]: answerIndex }));
152   sendAnswer(roomCode, studentEmail!, selectedOptionText);
153 };
154 const handleFinishGame = () => {
155   setIsGameActive(false);
156   setGameMode('report');
157   let highestScore = -1;
158   let currentWinner = null;
159   const currentScores = scoresRef.current;
160   const scoreEntries = Object.entries(currentScores);
161   console.log("Final Sync Scores:", currentScores);
162   console.log("Calculating Winner from Scores:", scoreEntries);
163   if (scoreEntries.length > 0) {
164     scoreEntries.forEach(([pId, score]) => {
165       const numericScore = Number(score);
166       const numericId = Number(pId);
167
168       if (numericScore > highestScore) {
169         highestScore = numericScore;
170         currentWinner = numericId;
171       }
172     });
173   }
174   console.log("Game Finished! Winner ID:", currentWinner, "with score:",
↪   highestScore);
175   setWinnerId(currentWinner);
176   setMasterAnswers(finalAnswers => {
177     const finalScore = currentQuestions.reduce((acc, q) => {
178       const selectedIdx = finalAnswers[q.questionID];

```

```

179     if (selectedIdx === undefined) return acc;
180     const options = getOptionsArray(q);
181     return options[selectedIdx] === q.correctAnswer ? acc + 10 : acc;
182 }, 0);
183 if (studentEmail) {
184     createBattleEntry(studentEmail, "COMPLETED", finalScore);
185 }
186 return finalAnswers;
187 });
188 };
189 const formatTime = (seconds: number) => `${Math.floor(seconds / 60)}:${(seconds %
↵ 60).toString().padStart(2, '0')}`;
190 if (gameMode === 'lobby') {
191     return (
192         <BattleLobby
193             roomCode={roomCode}
194             onStart={() =>
↵ fetch(`http://localhost:8080/api/battle/start/${roomCode}/${studentEmail}`, {
↵ method: 'POST' })}}
195             onCancel={() => setGameMode('menu')}
196             isHost={isHost}
197             players={players}
198         />
199     );
200 }
201 if (gameMode === 'join') return <JoinBattleLobby onJoin={handleJoinWithCode}
↵ onCancel={() => setGameMode('menu')} />;
202 if (gameMode === 'report') {
203     return (
204         <BattleReport
205             questions={currentQuestions}
206             selectedAnswers={masterAnswers}
207             timeTaken={Math.floor((Date.now() - startTime) / 1000)}
208             totalQuestions={currentQuestions.length}
209             winnerId={winnerId}
210             onBack={() => setGameMode('menu')}
211             onPlayAgain={() => {
212                 setGameMode('menu');
213                 setCurrentQuestion(0);
214                 setSelectedAnswers({});
215                 setMasterAnswers({});
216                 setShowResults({});
217             }}
218             players={players}
219         />
220     );
221 }
222 if (isComputerMode && gameMode === 'game') {

```

```

223   return (
224     <ComputerBattle
225       questions={currentQuestions}
226       studentId={studentId}
227       studentEmail={studentEmail}
228       firstName={user?.first_name}
229       onBack={() => {
230         setIsComputerMode(false);
231         setGameMode('menu');
232       }}
233     />
234   );
235 }
236
237 if (gameMode === 'game') {
238   const question = currentQuestions[currentQuestion];
239   if (!question) return null;
240   const options = getOptionsArray(question);
241   const isAnswered = showResults[question.questionID];
242   const isCorrect = options[selectedAnswers[question.questionID]] ===
↪ question.correctAnswer;
243   return (
244     <div className="min-h-screen bg-background pt-20 pb-16">
245       <div className="container mx-auto px-4 max-w-4xl">
246         <div className="flex justify-between items-center mb-4">
247           <div>
248             <Badge variant="secondary" className="mb-2"><Swords className="w-4 h-4
↪ mr-2" />Quiz Battle</Badge>
249             <h2 className="text-2xl font-bold">Question {currentQuestion + 1} of
↪ {currentQuestions.length}</h2>
250           </div>
251           <Card className="p-4 bg-gradient-card">
252             <div className="flex items-center gap-2"><Clock className="w-5 h-5
↪ text-primary" /><span className="text-2xl
↪ font-bold">{formatTime(timeLeft)}</span></div>
253           </Card>
254         </div>
255         <div className="grid grid-cols-2 md:grid-cols-4 gap-2 mb-6">
256           {players.map((pId) => (
257             <Card key={pId} className={`p-2 border-2 ${pId === studentId ?
↪ 'border-primary' : 'border-transparent'}`}>
258               <div className="flex justify-between items-center">
259                 <span className="text-xs font-bold truncate">Player
↪ {pId.toString().slice(-3)}</span>
260                 <Badge variant="secondary" className="bg-primary/10
↪ text-primary">
261                   {liveScores[pId.toString()] ?? liveScores[pId] ?? 0}
262                 </Badge>

```

```

263         </div>
264     </Card>
265     )}}
266 </div>
267     <Progress value={{(currentQuestion + 1) / currentQuestions.length) * 100}}
↪     className="h-2 mb-8" />
268     <Card className="p-8 bg-gradient-card mb-6">
269         <div className="flex items-start gap-4 mb-6">
270             <Badge variant="outline" className="text-lg px-3 py-1">{currentQuestion
↪             + 1}</Badge>
271             <h2 className="text-2xl font-semibold
↪             flex-1">{question.questionText}</h2>
272             </div>
273             <div className="space-y-3">
274                 {options.map((option, idx) => (
275                     <Button
276                         key={idx}
277                         variant="outline"
278                         onClick={() => !isAnswered && handleAnswer(idx)}
279                         disabled={isAnswered}
280                     >
281                         <span className="font-semibold mr-3 text-lg">{String.fromCharCode(65
↪                         + idx)}</span><span className="text-base">{option}</span>
282                     </Button>
283                 )}}
284             </div>
285         </Card>
286     </div>
287 </div>
288 );
289 }
290 return (
291     <div className="min-h-screen bg-background pt-20 pb-16">
292         <div className="container mx-auto px-4 text-center">
293             {gameMode === 'menu' && (
294                 <div>
295                     <Badge variant="secondary" className="mb-4"><Swords className="w-4 h-4
↪                     mr-2" />Quiz Battle Arena</Badge>
296                     <h1 className="text-4xl font-bold mb-6">Battle Mode Activated</h1>
297                     <div className="grid grid-cols-1 md:grid-cols-2 gap-8 mt-12">
298                         {battleModes.map((mode) => (
299                             <Card key={mode.id} className="p-6 bg-gradient-card group
↪                             hover:border-primary/50 transition-all">
300                                 <div className={`w-16 h-16 ${mode.color} rounded-full flex
↪                                 items-center justify-center mx-auto mb-4`} ><mode.icon className="w-8 h-8
↪                                 text-white" /></div>
301                                 <h3 className="text-xl font-semibold mb-4">{mode.title}</h3>
302                                 {mode.id === 2 ? (

```

```

303         <div className="space-y-2">
304             <Button onClick={handleInvite} className="w-full"><Users
↪   className="w-4 h-4 mr-2" />Invite Friends</Button>
305             <Button onClick={() => setGameMode('join')} variant="outline"
↪   className="w-full">Join Room</Button>
306         </div>
307     ) : (
308 <Button
309     onClick={async () => {
310         const res = await fetch(`http://localhost:8080/api/battle/create/${studentEmail}`,
↪   { method: 'POST' });
311         const code = await res.text();
312         const roomRes = await
↪   fetch(`http://localhost:8080/api/battle/join/${code}/${studentEmail}`, { method:
↪   'POST' });
313         const roomData = await roomRes.json();
314         setDbQuestions(roomData.questions);
315         setIsComputerMode(true);
316         setGameMode('game');
317     }}
318     className="w-full"
319 >
320     Start Battle
321 </Button>
322     )}
323 </Card>
324     )}
325 </div>
326 </div>
327     )}
328 </div>
329 </div>
330 );
331 };
332 export default Battle;

```

Chapter 5

Testing

5.1 Types of Testing

Software testing is the process of evaluating a software item to detect differences between the given input and expected output. Also, to assess the features of a software item. Testing assesses the quality of the product. Software testing is a verification and validation process.

1. Unit Testing:

It focuses on the smallest unit of software design. In this, we test an individual unit or group of interrelated units. It is often done by a programmer by using sample input and observing its corresponding outputs. After the development of the code, each developer tests the implemented code. The bugs and defects found in testing were then corrected.

2. Integration Testing:

The objective is to take unit-tested components and build a program structure that has been dictated by design. Integration testing is testing in which a group of components are combined to produce output. For integration testing, the changes made by all developers are merged into the application using a clear case, and the application is tested.

3. System Testing:

The software is tested such that it works fine for different operating systems. It is covered under the black box testing technique. In this, we just focus on the required input and output without focusing on the internal working. For System testing, the applications are tested for security and performance issues. After getting a green flag from this testing, the user acceptance testing is performed.

4. User acceptance testing:

User acceptance is performed by the Users of the application to certify the system with respect to the requirements that were agreed upon. When the user gives a green signal, the application changes are deployed on live servers.

Chapter 6

User Interface

6.1 Description of Components

The user interface of Hack Hustle is designed to be simple, interactive, and user-friendly for both students and faculty. The layout emphasises clarity and ease of navigation, ensuring that users can quickly access key features such as MCQ practice, quizzes, mock interviews, and progress tracking.

- **Student Interface:** Provides options to attempt subject-wise MCQs, take quizzes, view progress reports, and participate in AI-driven mock interviews.
- **Practise Module:** Multiple choice questions are displayed from various subjects and respective topics.
- **Interview Module:** The user is allowed to attempt an AI based interview based on their details provided along with detailed analysis.

6.2 Screen Layouts

1. Dashboard

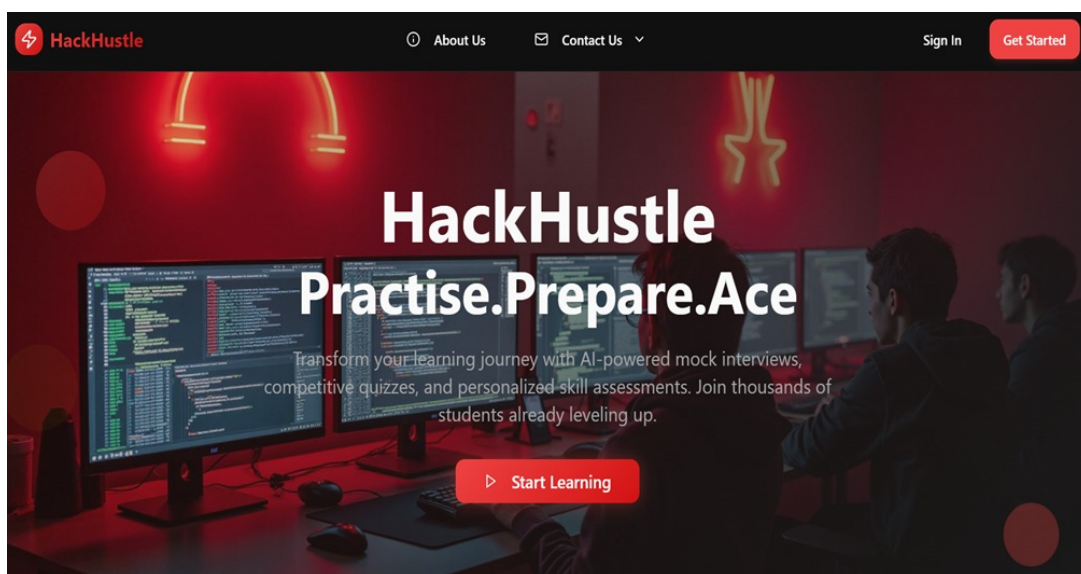


Figure 6.1: The landing page of the platform

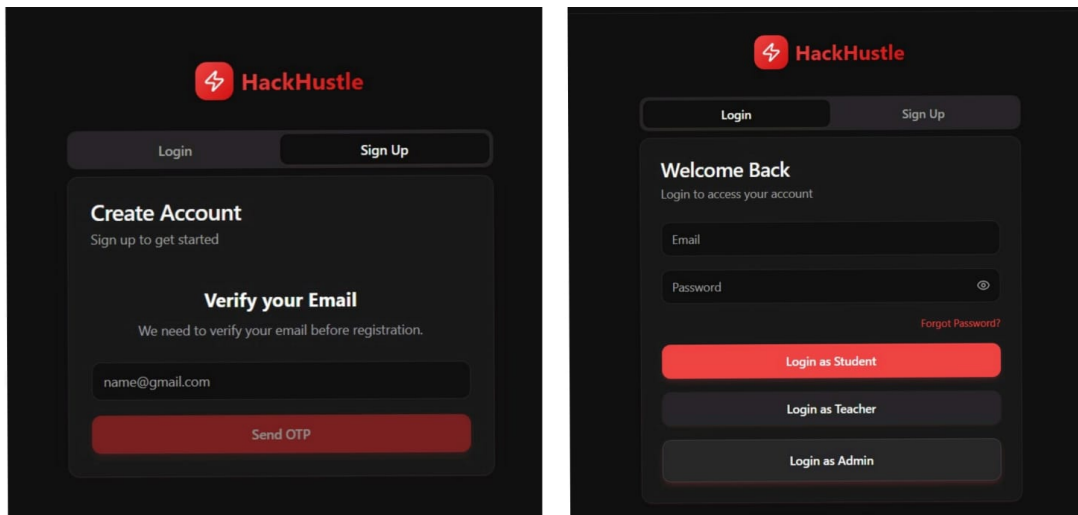


Figure 6.2: Login and Sign Up page of the platform

2. Practice Module Interface

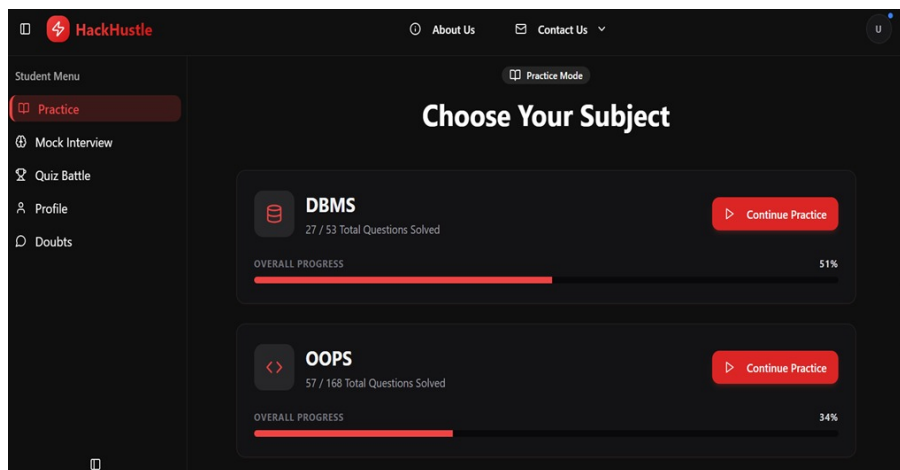


Figure 6.3: Practice page showing various subjects

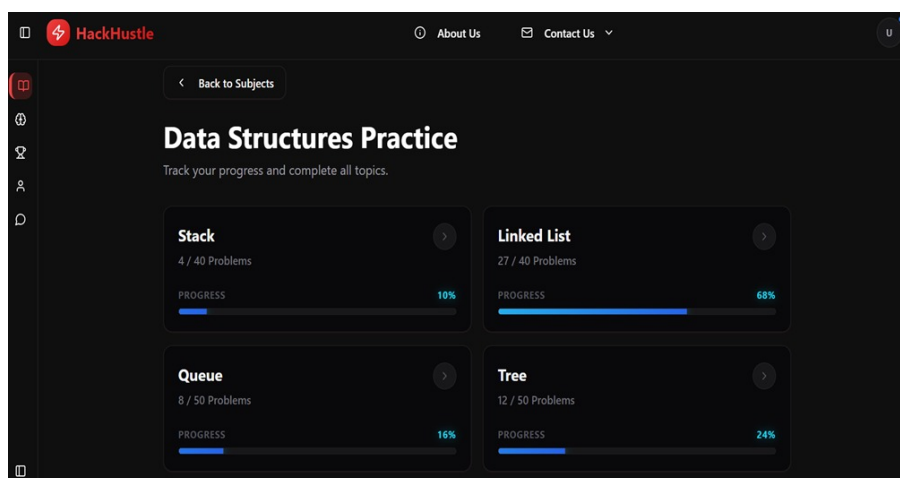


Figure 6.4: Various topics being listed under a subject

3. Interview Interface

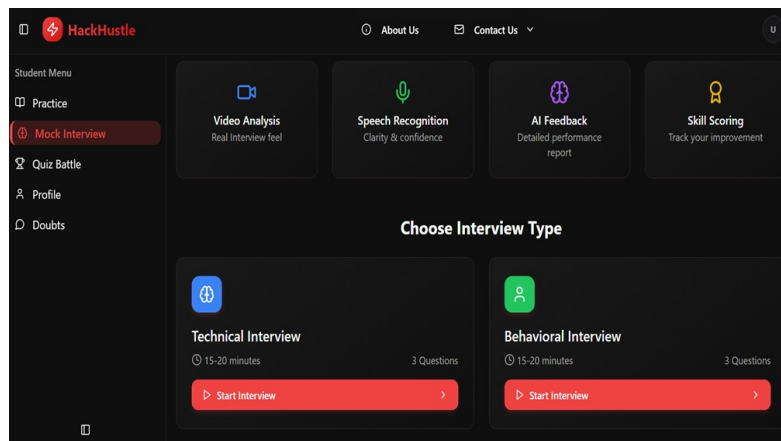


Figure 6.5: The Interview page showing technical and behavioral interview options

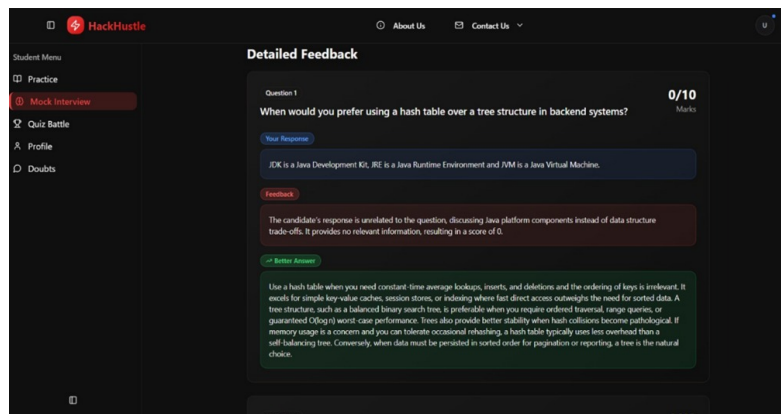


Figure 6.6: Analysis after the interview showing the question asked, the answer provided and better answer

4. Quiz Battle Interface

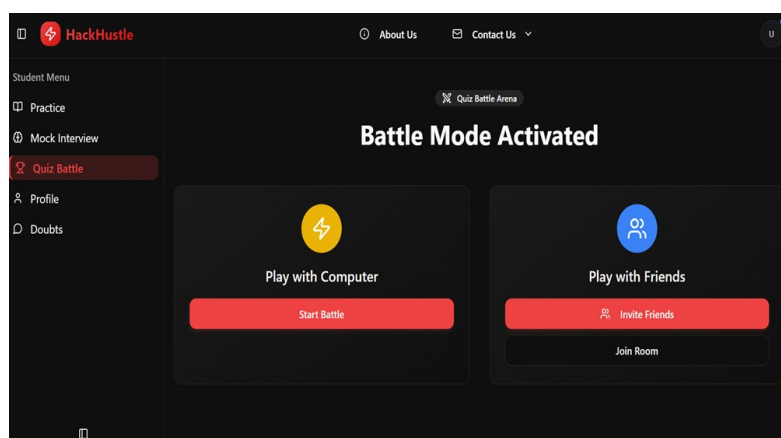


Figure 6.7: The Quiz Battle module showing modes to play a quiz, i.e. play with computer or a friend

5. Doubts Interface

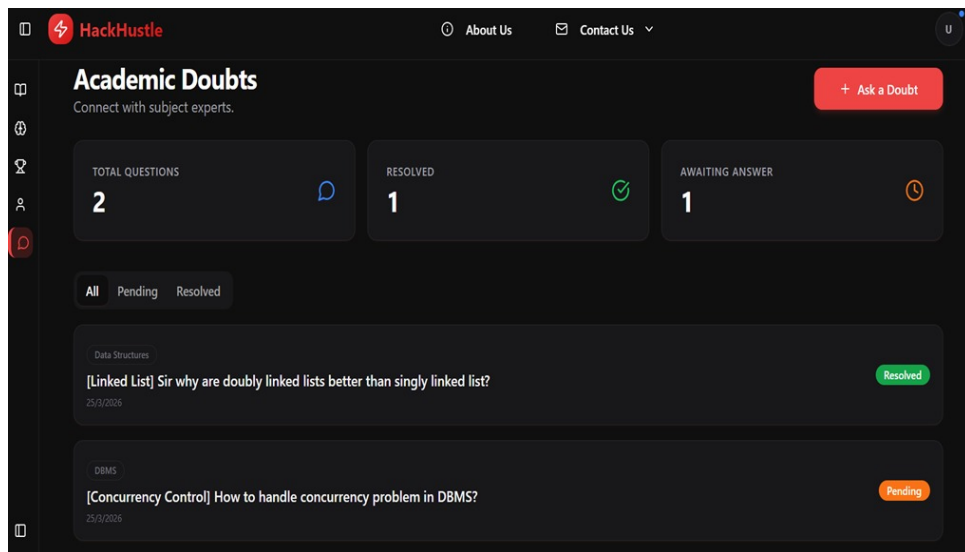


Figure 6.8: Student's doubt to be displayed on the Academic Doubts page

6. Teacher's Interface

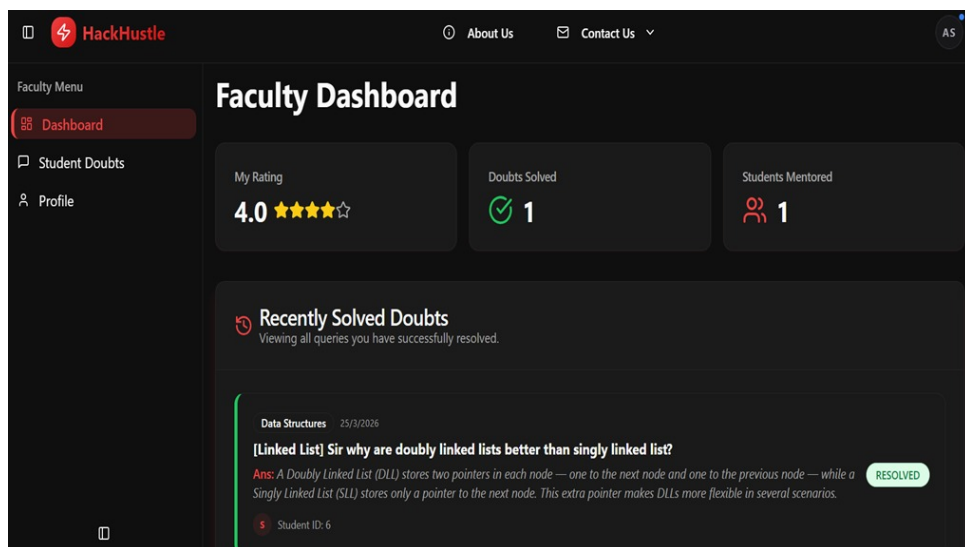


Figure 6.9: The doubts asked are listed on the Faculty Dashboard

Chapter 7

Appendix A: Glossary

7.1 Acronyms & Abbreviations

- **ADMIN:** The administrator is the website's controller, responsible for maintaining all records of functionalities and users.
- **GUEST USER:** People who can only browse through the website's home page but do not have access to full functionality.
- **REGISTERED USER:** A person who has registered himself/herself by signing up allowing him/her to access the full range of features offered by the platform.
- **DATABASE:** A collection of interrelated data that contains information required by the system.
- **IEEE:** Institute of Electrical and Electronics Engineers
- **WWW:** World Wide Web
- **SERVER:** The main computer on the network
- **BROWSER:** A software application used to locate and display web pages.
- **SRS:** Software Requirement Specification
- **SDS:** Software Design Specification
- **HTTP:** Hypertext Transfer Protocol is a transaction-oriented client/server protocol between a web browser and a web server.
- **HTTPS:** Secure Hypertext Transfer Protocol is an HTTP over SSL (Secure Socket Layer).
- **RAM:** Random Access Memory

Chapter 8

References

- IEEE Recommended Practice for Software Requirements Specification- IEEE Std. 830-1993.
- Pressman, Roger S., Software Engineering: A Practitioner's Approach, Fifth Edition, McGraw-Hill Publication, 2000.